



EMBEDDED SYSTEM DESIGN AND IoT

25EC2206E

STUDENT ID:
STUDENT NAME:

ACADEMIC YEAR: 2026-27

Table of Contents

Embedded System Design and IoT Laboratory – Session Plan

Session No	Topics	Page No
1	Installation and Configuration of STM32CubeIDE, STM32MX IDE and Creation of the First STM32 Project	
2	Design and Implementation of an STM32-Based LED Blinking System with Dynamic Clock Configuration and Performance Analysis	
3	Development of a SysTick Timer-Based Periodic LED Task Scheduling System without Using Software Delays	
4	Constructing a Debounced GPIO Input Interface: Push Button Controlled LED Operation	
5	Evaluating GPIO Drive Strengths Performance Analysis of GPIO Speed and Pull-Up/Pull-Down Configurations	
6	Implementing Logic-Level Control for High-Power Relay Interfacing and Digital Output Control Using STM32	
7	Design and Implementation of a 7-Segment Display Interface Using STM32	
8	Interfacing an I ² C 16×2 LCD and Displaying User Information	
9	Developing and Analyzing UART Serial Communication: Data Transmission from STM32 to PC	
10	Developing and Analyzing UART Serial Communication: Data Transmission from PC to STM32	
11	Development of a UART Echo Application Using STM32 display in LCD	
12	Designing a PWM Signal Generation Using STM32 Timers	
13	Designing Asynchronous Data Interrupt-Driven UART Communication Using HAL Library	
14	Design and Development of a PWM-Controlled Stepper Motor System Using STM32	
15	Designing a Servo Motor Position Control Using PWM	
16	Design and Implementation of a Bidirectional PWM-Controlled DC Motor Drive Using an H-Bridge Driver	
17	Designing a Single-Channel ADC Interfacing Temperature Sensor	
18	Displaying LDR Light Intensity on an I ² C LCD	
19	Developing a CAN Peripheral Initialization and Loopback Communication Test	
20	Developing a CAN Communication Between Two STM32 Nucleo Boards	
21	Designing a Sensor Data Acquisition and Transmission Over CAN Bus	
22	Project Phase-I: System Requirements Analysis and Hardware Design	
23	Project Phase-II: Peripheral Configuration and Software Development	
24	Project Phase-III: Final System Testing, Performance Evaluation, and Demonstration	

A.Y. 2026-2027 LAB CONTINUOUS EVALUATION

LIST OF EXPERIMENTS

Exp. No	Date	Experiment Title	Project Planning & Feasibility (10M)	System Design & Architecture (10M)	Implementation & Prototype Development (10M)	Testing, Validation & Documentation (10M)	Technical Presentation & Viva Voce (10M)	Total (50M)	Faculty Signature
1		Installation and Configuration of STM32CubeIDE and Creation of the First STM32 Project							
2		Implementing and Analyzing GPIO Programming: Blinking the Onboard LED Using STM32 HAL							
3		Development of a SysTick Timer-Based Periodic LED Task Scheduler Without Using Software Delays							
4		Constructing a Debounced GPIO Input Interface: Push Button Controlled LED Operation							
5		Evaluating GPIO Drive Strengths — Performance Analysis of GPIO Speed and Pull-Up/Pull-Down Configurations							
6		Implementing Logic-Level Control for High-Power Relay Interfacing and Digital Output Control Using STM32							
7		Interfacing an I ² C 16×2 LCD and Displaying User Information with Relay Status							
8		Developing and Analyzing UART Serial Communication: Data Transmission from STM32 to PC							
9		Developing and Analyzing UART Serial Communication: Data Reception from PC to STM32							
10		Development of a UART Echo Application Using STM32 Display in LCD							
11		Designing Asynchronous Data Interrupt-Driven UART Communication Using HAL Library							
12		Synthesizing Data Streams from UART to I ² C Peripheral Displays							
13		Developing a Rule-Based Automated Control System Using UART Communication							
14		Designing a PWM Signal Generation Using STM32 Timers							
15		Designing a Servo Motor Position Control Using PWM							
16		Design and Implementation of a Bidirectional PWM-Controlled DC Motor Drive Using an H-Bridge Driver							
17		Designing a UART-Based Command-Controlled Servo Motor Positioning							
18		Designing a Single-Channel ADC Interfacing for Potentiometer Voltage Measurement							
19		Designing a Multi-Channel ADC Interfacing for LM35 Temperature Sensor Measurement							
20		Multi-Channel ADC Interfacing for LDR-Based Ambient Light Measurement							
21		Displaying LM35 Temperature and LDR Light Intensity on an I ² C LCD							
22		Intelligent Environmental Control: Relay Switching Based on LDR and DC Motor Speed Control Based on LM35							
23		Monitoring and Transmission of Sensor Data (LM35 & LDR) to PC with Actuator Status Display							
24		Developing a CAN Peripheral Initialization and Loopback Communication Test							
25		Developing a CAN Communication Between Two STM32 Nucleo Boards							
26		Designing a Sensor Data Acquisition and Transmission Over CAN Bus							
27		Project Phase-I: System Requirements Analysis and Hardware Design							
28		Project Phase-II: Peripheral Configuration and Software Development							

Exp. No	Date	Experiment Title	Project Planning & Feasibility (10M)	System Design & Architecture (10M)	Implementation & Prototype Development (10M)	Testing, Validation & Documentation (10M)	Technical Presentation & Viva Voce (10M)	Total (50M)	Faculty Signature
29		Project Phase-III: CAN Communication Integration and Testing							
30		Project Phase-IV: Sensor and Actuator Integration							
31		Project Phase-V: Final System Testing, Performance Evaluation, and Demonstration							

Note: Marks columns are filled in by the evaluator for each experiment as it is completed.

Experiment 1

Installation and Configuration of STM32CubeIDE and Creation of the First STM32 Project

Aim/Objective: To install and configure STM32CubeIDE, and to create, build, and flash a first working project on the Nucleo-F446RE development board.

PRE-REQUISITE:

1. Basic C programming knowledge.
2. Familiarity with installing and running desktop software on Windows/Linux/macOS.

Tools & Components Required:

- STM32CubeIDE, STM32CubeMX & STM32 HAL Library
- ST Nucleo-F446RE (STM32F446RET6, Arm Cortex-M4 @ 180 MHz)
- USB Type-A to Mini-B/Micro-B cable (ST-LINK)
- Host PC (Windows/Linux/macOS)

Description:

STM32CubeIDE is ST Microelectronics' free, Eclipse-based Integrated Development Environment that bundles a C/C++ compiler (GCC ARM), a debugger (GDB with ST-LINK GDB server), and the STM32CubeMX graphical configuration tool in a single application. STM32CubeMX allows the pin-out, clock tree, and middleware of an STM32 microcontroller to be configured graphically and then generates initialization C code built on the STM32 HAL (Hardware Abstraction Layer).

The Nucleo-F446RE integrates an on-board ST-LINK/V2-1 debugger/programmer, so no external programmer is required -- a single USB cable provides power, programming, and a virtual COM port (VCP) for serial communication with the PC.

Circuit / Pin Connections:

Signal	Nucleo-F446RE Pin	Notes
Board Power / Programming	USB (CN1)	Connects ST-LINK to host PC; also powers the board

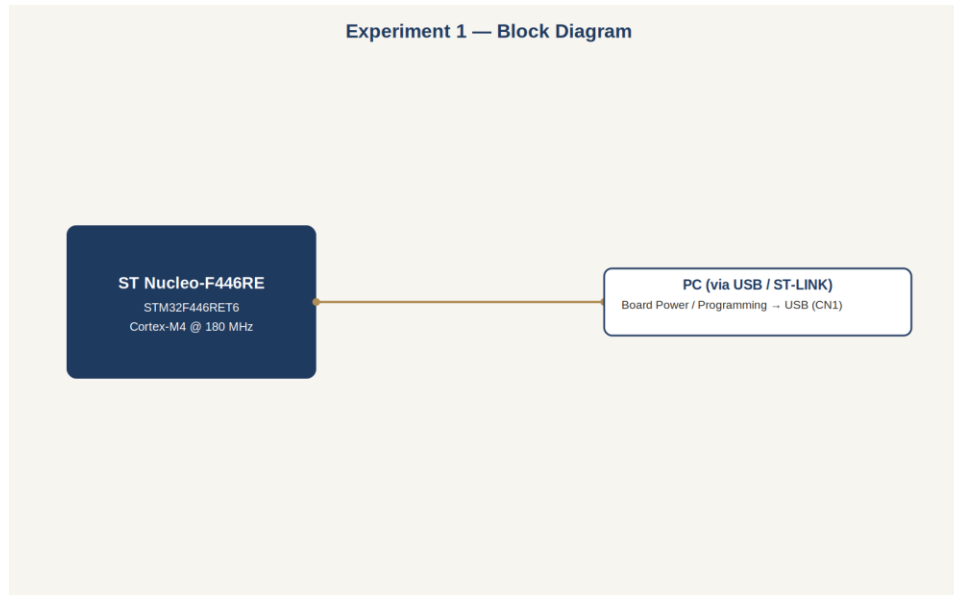


Figure: Block diagram for Experiment 1

Procedure:

Configure the required peripherals in STM32CubeMX and implement a HAL-based program to achieve the objective stated above, following the procedure and pin connections given below. Build, flash, and verify the functionality on the Nucleo-F446RE hardware.

1. Download and install STM32CubeIDE from st.com (Windows/Linux/macOS installer).
2. Launch STM32CubeIDE and select or create a workspace directory.
3. File > New > STM32 Project. In the MCU/Board Selector, choose the 'Board Selector' tab and search for 'NUCLEO-F446RE'.
4. Select the board, keep 'Initialize all peripherals with their default mode' checked, and click Next.
5. Enter a project name (e.g. Lab01_FirstProject), keep Targeted Language = C, Targeted Binary Type = Executable, and click Finish.
6. Accept the prompt to initialize peripherals in default mode; the Pinout & Configuration view (CubeMX perspective) opens automatically.
7. Click the .ioc file's 'Generate Code' button (or Project > Generate Code) to generate the initialization source files.
8. Switch to the C/C++ perspective, open Core/Src/main.c, and note the auto-generated SystemClock_Config() and MX_GPIO_Init() calls.
9. Connect the Nucleo board via USB; verify it is detected (ST-LINK drivers install automatically on most systems).
10. Click the Build (hammer) icon to compile, then Run (or Debug) to flash the .elf binary onto the microcontroller over SWD.

Program:

Viva-voce (In-Lab):

1. What is the role of STM32CubeMX inside STM32CubeIDE?

2. Which files does STM32CubeMX regenerate versus preserve when code is re-generated after a pinout change?

3. What debugging protocol does the on-board ST-LINK/V2-1 use to communicate with the STM32F446RE?

4. Why is HAL_Init() the first call inside main()?

Post Lab Requirements:

1. Save the screenshots/pictures of the output obtained and bring the same to the next lab.
2. Note the procedures and observations as instructed by your advisor.
3. Submit the practical report in the next lab for correction.

Results

STM32CubeIDE installs successfully, the board is recognized by the ST-LINK utility, and the generated project builds without errors. After flashing, the debugger halts at main(), confirming the toolchain, drivers, and board are correctly configured end to end.

Analysis and Inferences:

Evaluator Remark (if Any):	Marks Secured: _____ out of 50
	Signature of the Evaluator with Date

Evaluator MUST ask Viva-voce prior to signing and posting marks for each experiment.

Experiment 2

Implementing and Analyzing GPIO Programming: Blinking the Onboard LED Using STM32 HAL

Aim/Objective: To configure a GPIO pin as a digital output using STM32CubeMX/HAL and to blink the on-board user LED (LD2) at a fixed rate, analyzing the HAL GPIO API in the process.

PRE-REQUISITE:

1. Completion of Experiment 1 (STM32CubeIDE installation and first project).
2. Basic understanding of digital logic (HIGH/LOW states).

Tools & Components Required:

- STM32CubeIDE, STM32CubeMX & STM32 HAL Library
- ST Nucleo-F446RE (STM32F446RET6, Arm Cortex-M4 @ 180 MHz)
- USB cable

Description:

General Purpose Input/Output (GPIO) pins are the most fundamental peripheral on any microcontroller. Each pin can be independently configured as an input, output, or alternate function, with configurable speed, pull-up/pull-down, and output type (push-pull or open-drain). The STM32 HAL exposes this through the GPIO_InitTypeDef structure and HAL_GPIO_Init().

On the Nucleo-F446RE, the on-board green LED LD2 is wired to pin PA5, and can be toggled entirely in software without any external circuitry, making it the canonical 'hello world' of embedded programming.

Circuit / Pin Connections:

Signal	Nucleo-F446RE Pin	Notes
LD2 (on-board LED)	PA5	Output push-pull, driven directly by MCU; no external wiring needed

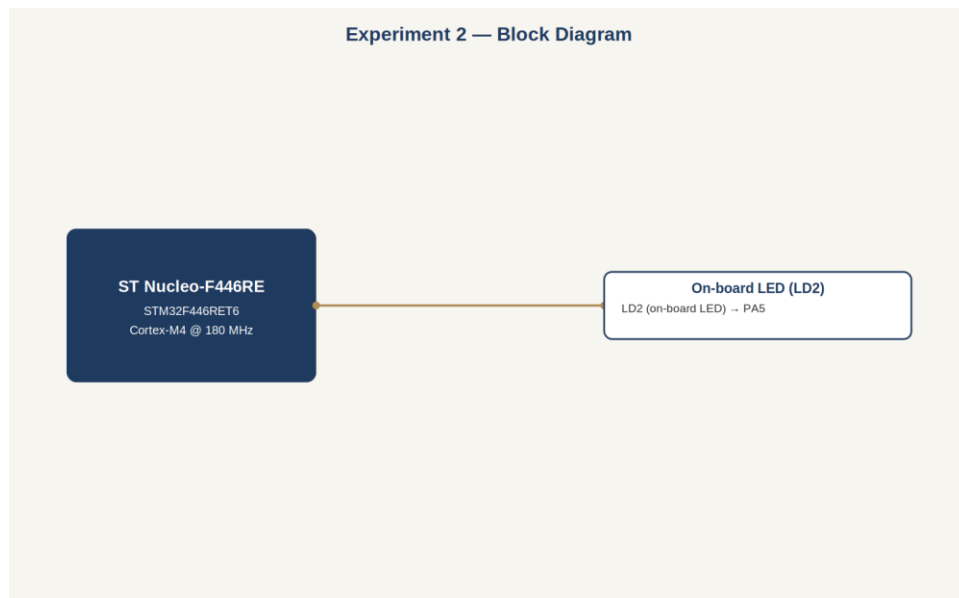


Figure: Block diagram for Experiment 2

Procedure:

Configure the required peripherals in STM32CubeMX and implement a HAL-based program to achieve the objective stated above, following the procedure and pin connections given below. Build, flash, and verify the functionality on the Nucleo-F446RE hardware.

1. Create a new STM32CubeIDE project targeting NUCLEO-F446RE (or reuse Lab01).
2. In the Pinout view, click pin PA5 and set it to GPIO_Output (it is pre-labelled LD2 by the board support).
3. In the GPIO configuration panel, set GPIO output level = Low, Output push-pull, No pull-up/down, Speed = Low.
4. Generate the code and open main.c.
5. Inside the while(1) loop, call HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5) followed by HAL_Delay(500).
6. Build and flash the project; observe LD2 blinking at a 1 Hz rate (500 ms on, 500 ms off).
7. Modify the HAL_Delay() argument to change the blink rate and re-flash to confirm the timing changes accordingly.

Program:

Viva-voce (In-Lab):

1. What is the difference between HAL_GPIO_WritePin() and HAL_GPIO_TogglePin()?

2. Why must a GPIO pin used to drive an LED be configured as push-pull rather than open-drain?

3. What mechanism does HAL_Delay() use internally to measure time?

4. What would happen to the blink rate if the system clock configuration were changed without updating SysTick?

Post Lab Requirements:

1. Save the screenshots/pictures of the output obtained and bring the same to the next lab.
2. Note the procedures and observations as instructed by your advisor.
3. Submit the practical report in the next lab for correction.

Results

LD2 blinks continuously at approximately 1 Hz. Because HAL_Delay() blocks the CPU using the SysTick interrupt count, the LED period exactly tracks the requested delay, confirming correct GPIO configuration and clock setup.

Analysis and Inferences:

Evaluator Remark (if Any):	Marks Secured: _____ out of 50
	Signature of the Evaluator with Date

Evaluator MUST ask Viva-voce prior to signing and posting marks for each experiment.

Experiment 3

Development of a SysTick Timer-Based Periodic LED Task Scheduler Without Using Software Delays

Aim/Objective: To replace blocking HAL_Delay() calls with a non-blocking, SysTick-driven millisecond tick counter that schedules a periodic LED task, demonstrating the foundation of cooperative task scheduling.

PRE-REQUISITE:

1. Completion of Experiment 2 (GPIO output and HAL_Delay basics).
2. Basic understanding of timers and interrupts in microcontrollers.

Tools & Components Required:

- STM32CubeIDE, STM32CubeMX & STM32 HAL Library
- ST Nucleo-F446RE (STM32F446RET6, Arm Cortex-M4 @ 180 MHz)
- USB cable

Description:

HAL_Delay() blocks CPU execution for its entire duration, which is unacceptable in applications that must service multiple tasks concurrently (e.g. reading sensors while blinking an LED). The Cortex-M SysTick timer generates a periodic interrupt (HAL configures it for 1 ms by default) that increments a global millisecond counter, HAL_GetTick().

A simple non-blocking scheduler compares the elapsed time (HAL_GetTick() - lastTick) against a desired period on every iteration of the main loop, executing the task only when the period has elapsed -- allowing other code to run in between without ever blocking.

Circuit / Pin Connections:

Signal	Nucleo-F446RE Pin	Notes
LD2 (on-board LED)	PA5	Toggled by the non-blocking scheduler

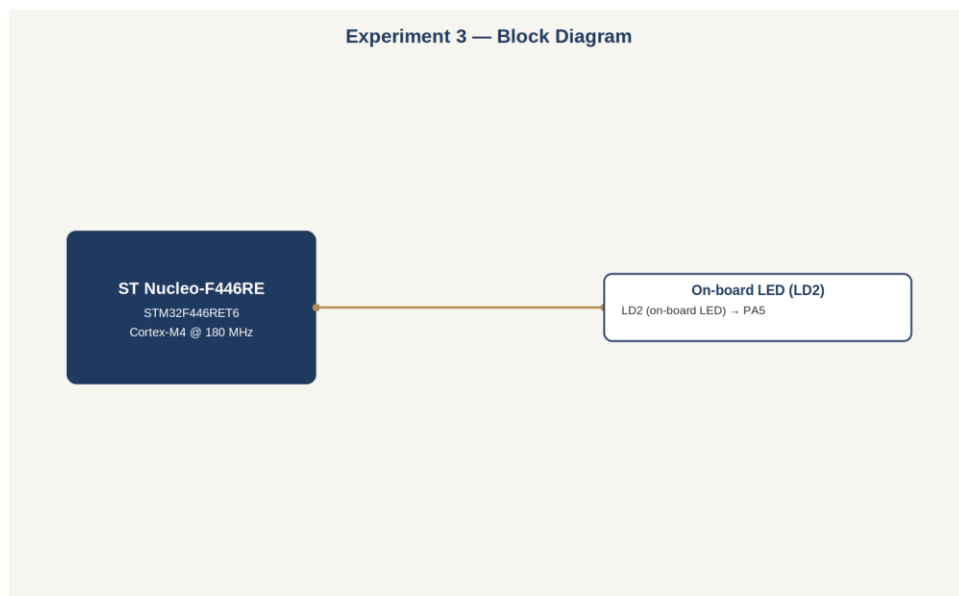


Figure: Block diagram for Experiment 3

Procedure:

Configure the required peripherals in STM32CubeMX and implement a HAL-based program to achieve the objective stated above, following the procedure and pin connections given below. Build, flash, and verify the functionality on the Nucleo-F446RE hardware.

1. Reuse the GPIO configuration for LD2 (PA5) from Experiment 2.
2. In main.c, declare a static uint32_t variable, e.g. lastToggleTick = 0, before the while(1) loop.
3. Inside the loop, compare HAL_GetTick() - lastToggleTick against the desired interval (e.g. 500 ms).
4. When the interval has elapsed, toggle LD2 and update lastToggleTick = HAL_GetTick().
5. Add a second independent 'task' (e.g. incrementing a counter every 100 ms) to demonstrate that both tasks now run concurrently without either blocking the other.
6. Build, flash, and observe the LED blinking identically to Experiment 2 while the CPU remains free between toggles.

Program:

Viva-voce (In-Lab):

1. Why does the subtraction (now - lastToggleTick) work correctly even when HAL_GetTick() eventually overflows (wraps around)?

2. What is the default SysTick interrupt period configured by HAL_Init(), and which register controls it?

3. How would you extend this scheduler to run three independent periodic tasks with different periods?

4. What are the limitations of this cooperative polling approach compared to a true RTOS?

Post Lab Requirements:

1. Save the screenshots/pictures of the output obtained and bring the same to the next lab.
2. Note the procedures and observations as instructed by your advisor.
3. Submit the practical report in the next lab for correction.

Results

LD2 blinks at the same visible rate as with HAL_Delay(), but the main loop is now free to execute other polling tasks in between toggles, since the CPU is never blocked waiting. Measured with a logic analyzer, the toggle jitter is bounded by the 1 ms SysTick resolution.

Analysis and Inferences:

Evaluator Remark (if Any):	Marks Secured: _____ out of 50
	Signature of the Evaluator with Date

Evaluator MUST ask Viva-voce prior to signing and posting marks for each experiment.

Experiment 4

Constructing a Debounced GPIO Input Interface: Push Button Controlled LED Operation

Aim/Objective: To configure a GPIO pin as a digital input, read the on-board push button (B1), and implement software debouncing so that each physical press toggles the on-board LED exactly once.

PRE-REQUISITE:

1. Completion of Experiments 2-3 (GPIO output and SysTick-based timing).
2. Basic understanding of GPIO digital input configuration.

Tools & Components Required:

- STM32CubeIDE, STM32CubeMX & STM32 HAL Library
- ST Nucleo-F446RE (STM32F446RET6, Arm Cortex-M4 @ 180 MHz)
- USB cable

Description:

Mechanical push buttons do not produce a single clean transition; contact bounce causes the input to oscillate between logic states for a few milliseconds after each press or release. Reading the raw pin state without debouncing can register a single press as multiple events.

A simple and effective software debounce technique samples the pin, and only accepts a new stable state after it has remained unchanged for a minimum hold time (typically 20-50 ms), measured using HAL_GetTick() in the same non-blocking style as Experiment 3.

Circuit / Pin Connections:

Signal	Nucleo-F446RE Pin	Notes
B1 (user push button)	PC13	Active LOW, external pull-up already present on the Nucleo board
LD2 (on-board LED)	PA5	Toggled once per debounced press

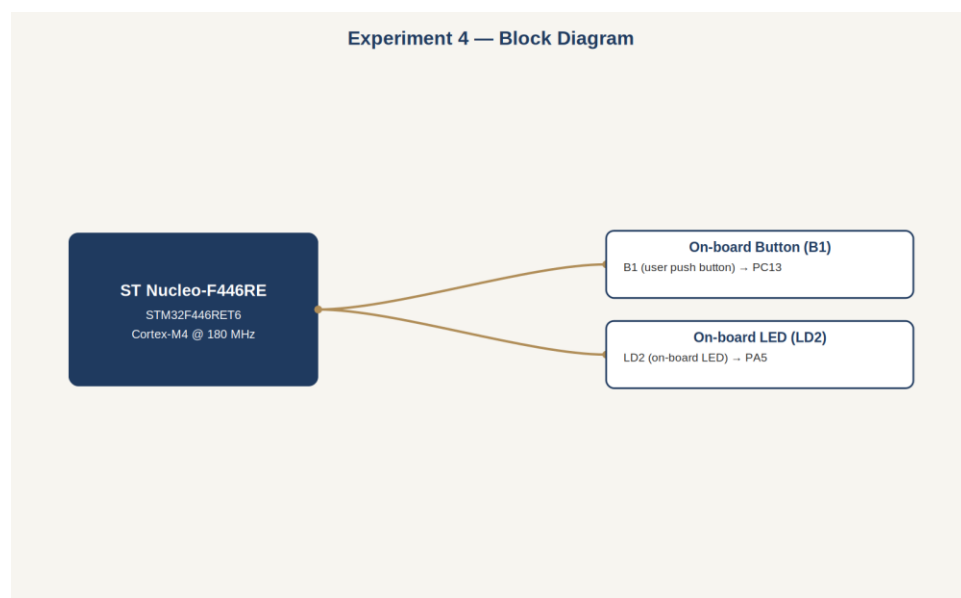


Figure: Block diagram for Experiment 4

Procedure:

Configure the required peripherals in STM32CubeMX and implement a HAL-based program to achieve the objective stated above, following the procedure and pin connections given below. Build, flash, and verify the functionality on the Nucleo-F446RE hardware.

1. In CubeMX, pin PC13 is pre-configured as GPIO_EXTI13 (B1) by the board support; leave it as a simple GPIO input for this polled version, with No pull-up/down (external pull-up already fitted).
2. Configure PA5 as GPIO_Output for LD2 as in Experiment 2.
3. In main.c, maintain a state variable `lastButtonState = GPIO_PIN_SET` (button released, active-low) and a `lastDebounceTick`.
4. On each loop iteration, read `HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_13)`; if it differs from `lastButtonState`, reset `lastDebounceTick = HAL_GetTick()`.
5. If `(HAL_GetTick() - lastDebounceTick)` exceeds the debounce interval (e.g. 30 ms) and the new reading is stable and LOW (pressed) while the previously accepted state was HIGH, toggle LD2 and update the accepted state.
6. Build, flash, and press B1 repeatedly, confirming a single, reliable LED toggle per press with no double-triggering.

Program:

Viva-voce (In-Lab):

1. What causes mechanical contact bounce, and what is a typical bounce duration for a tactile push button?

2. Why is the debounce interval measured with HAL_GetTick() rather than HAL_Delay()?

3. What is the alternative to software debouncing at the hardware level (e.g. RC filter)?

4. Why is B1 active-low on the Nucleo-F446RE, and how does that affect the polarity check in the code?

Post Lab Requirements:

1. Save the screenshots/pictures of the output obtained and bring the same to the next lab.
2. Note the procedures and observations as instructed by your advisor.
3. Submit the practical report in the next lab for correction.

Results

Without debouncing, LD2 occasionally toggles two or three times per physical press due to contact bounce. With the debounce filter enabled, each press reliably produces exactly one toggle, verified over at least 20 repeated presses.

Analysis and Inferences:

Evaluator Remark (if Any):	Marks Secured: _____ out of 50
	Signature of the Evaluator with Date

Evaluator MUST ask Viva-voce prior to signing and posting marks for each experiment.

Experiment 5

Evaluating GPIO Drive Strengths: Performance Analysis of GPIO Speed and Pull-Up/Pull-Down Configurations

Aim/Objective: To experimentally evaluate how the STM32 GPIO output-speed setting and internal pull-up/pull-down configuration affect signal rise/fall time, drive capability, and floating-input behaviour.

PRE-REQUISITE:

1. Completion of Experiments 2-4 (GPIO output/input basics).
2. Basic understanding of electrical concepts such as rise time and EMI.

Tools & Components Required:

- STM32CubeIDE, STM32CubeMX & STM32 HAL Library
- ST Nucleo-F446RE (STM32F446RET6, Arm Cortex-M4 @ 180 MHz)
- Oscilloscope or logic analyzer
- 10 kOhm resistor
- Breadboard and jumper wires
- LED with 330 ohm series resistor (optional external load)

Description:

Each STM32 GPIO pin can be configured with one of several output speed classes (Low, Medium, High, Very High), which trade switching speed against EMI and current consumption by controlling the output driver's slew rate. Faster settings produce steeper edges but higher instantaneous current and more electromagnetic emission.

Internal weak pull-up/pull-down resistors (typically 30-50 kOhm) can be enabled per pin to define a default logic level for floating inputs; disabling them on an unconnected input leaves the pin susceptible to noise-induced random toggling.

Circuit / Pin Connections:

Signal	Nucleo-F446RE Pin	Notes
Test output pin	PA5 (LD2) or PA6	Configured at different GPIO_SPEED_FREQ_* levels for scope measurement
Floating input test pin	PB0	Alternately configured with no-pull, pull-up, and pull-down

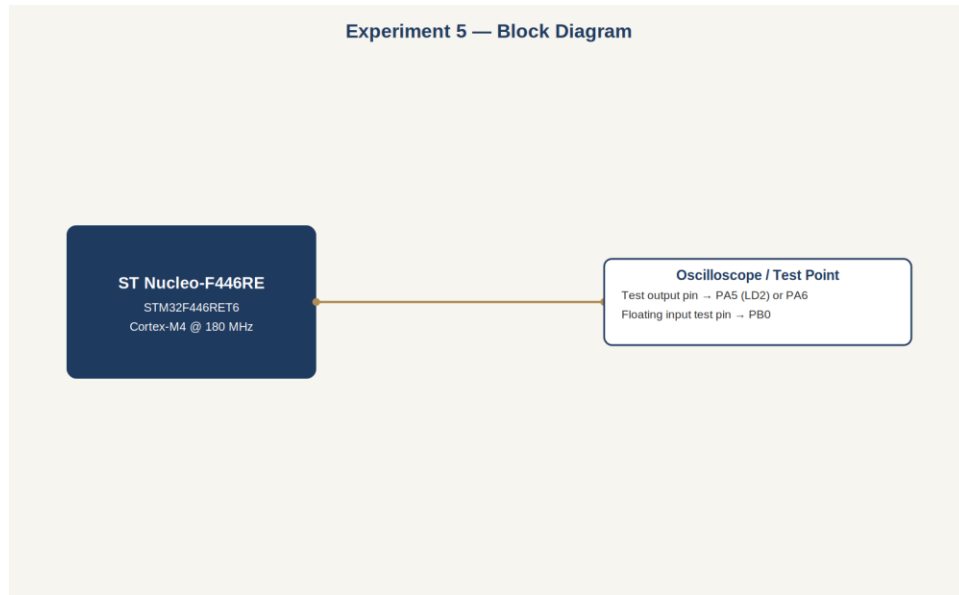


Figure: Block diagram for Experiment 5

Procedure:

Configure the required peripherals in STM32CubeMX and implement a HAL-based program to achieve the objective stated above, following the procedure and pin connections given below. Build, flash, and verify the functionality on the Nucleo-F446RE hardware.

1. Configure PA6 as GPIO_Output and connect an oscilloscope probe to it (with an external LED+resistor load for a realistic current draw).
2. In firmware, toggle PA6 in a tight loop and capture the rising-edge slew rate with GPIO_SPEED_FREQ_LOW selected; record the 10%-90% rise time.
3. Recompile with GPIO_SPEED_FREQ_VERY_HIGH selected for the same pin and repeat the capture; compare rise times.
4. Configure PB0 as a floating input (GPIO_NOPULL) with nothing connected; read and log HAL_GPIO_ReadPin() over UART or watch it in the debugger -- observe random/noisy transitions.
5. Reconfigure PB0 with GPIO_PULLUP enabled (still unconnected) and confirm the pin now reads a stable HIGH.
6. Reconfigure PB0 with GPIO_PULLDOWN enabled and confirm it now reads a stable LOW.
7. Tabulate rise time vs speed setting and input stability vs pull configuration.

Program:

Viva-voce (In-Lab):

1. Why does a faster GPIO speed setting increase electromagnetic interference?

2. What is the typical resistance range of the STM32 internal weak pull resistors?

3. In what practical scenario would you deliberately choose GPIO_SPEED_FREQ_LOW over Very High for an output pin?

4. Why is an unconnected (floating) digital input dangerous to leave unconfigured in a real design?

Post Lab Requirements:

1. Save the screenshots/pictures of the output obtained and bring the same to the next lab.
2. Note the procedures and observations as instructed by your advisor.
3. Submit the practical report in the next lab for correction.

Results

Rise time decreases markedly as the speed setting increases (typically tens of ns at Very High versus several hundred ns at Low, load-dependent). With GPIO_NOPULL and nothing connected, PB0 shows erratic logic-level readings; enabling the internal pull-up or pull-down produces a rock-stable HIGH or LOW respectively.

Analysis and Inferences:

Evaluator Remark (if Any):	Marks Secured: _____ out of 50
	Signature of the Evaluator with Date

Evaluator MUST ask Viva-voce prior to signing and posting marks for each experiment.

Experiment 6

Implementing Logic-Level Control for High-Power Relay Interfacing and Digital Output Control Using STM32

Aim/Objective: To interface a 5 V single-channel relay module to the Nucleo-F446RE and switch a high-power/high-voltage load (e.g. a lamp) safely under GPIO control.

PRE-REQUISITE:

1. Completion of Experiments 2 and 4 (GPIO output control and debounced input).
2. Basic understanding of transistor-based switching circuits.

Tools & Components Required:

- STM32CubeIDE, STM32CubeMX & STM32 HAL Library
- ST Nucleo-F446RE (STM32F446RET6, Arm Cortex-M4 @ 180 MHz)
- 5 V single-channel relay module (opto-isolated, with on-board driver transistor)
- External 5 V supply for the relay coil (or 5V pin from Nucleo)
- AC/DC load to be switched (e.g. lamp + socket) wired through the relay's NO/COM contacts
- Jumper wires

Description:

STM32 GPIO pins can source/sink only a few mA at 3.3 V logic and cannot directly drive an electromechanical relay coil or a high-power/high-voltage load. A relay module solves this with an on-board NPN transistor (or opto-isolator) that switches the relay coil current from a separate 5 V supply, using only a small 3.3 V logic-level signal from the MCU as the control input.

The relay's Normally-Open (NO) and Common (COM) contacts are electrically isolated from the control side, allowing the STM32 to safely switch mains-referenced or high-current loads without exposing the microcontroller to that voltage/current directly.

Circuit / Pin Connections:

Signal	Nucleo-F446RE Pin	Notes
Relay IN (control)	PB0	GPIO output, push-pull; most modules are active-LOW
Relay VCC	5V (Nucleo CN7 pin 18)	Powers the relay coil driver
Relay GND	GND	Common ground with the Nucleo
Load	Relay COM / NO terminals	External load and its own supply, isolated from MCU

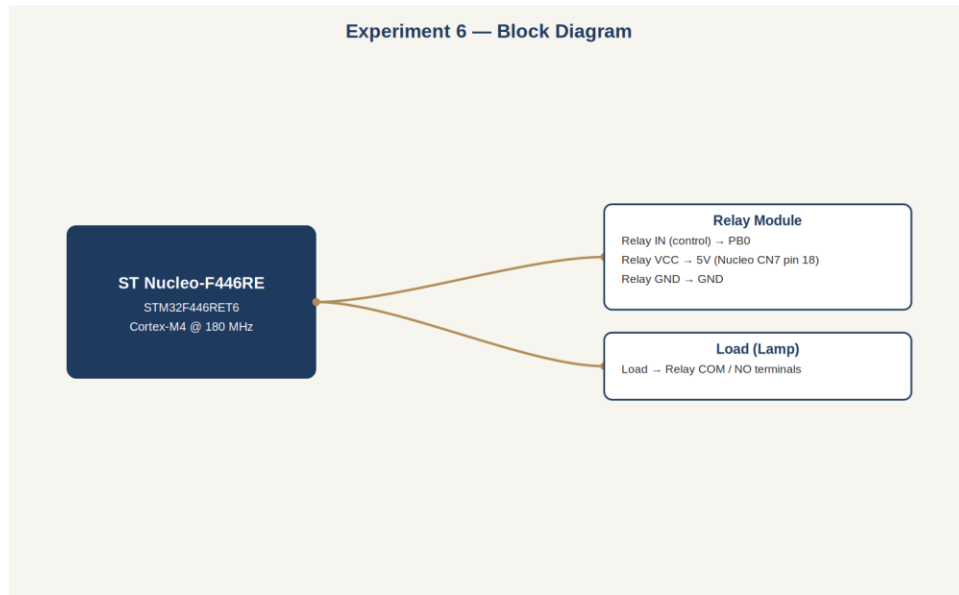


Figure: Block diagram for Experiment 6

Procedure:

Configure the required peripherals in STM32CubeMX and implement a HAL-based program to achieve the objective stated above, following the procedure and pin connections given below. Build, flash, and verify the functionality on the Nucleo-F446RE hardware.

1. Wire the relay module's VCC and GND to the Nucleo's 5V and GND pins, and its IN pin to PB0.
2. Wire the load (lamp) and its own power source in series through the relay's COM and NO terminals; double-check polarity/voltage ratings before applying power.
3. In CubeMX, configure PB0 as GPIO_Output, push-pull, no pull, low speed.
4. In main.c, drive PB0 LOW to energize the relay (for an active-low module) and HIGH to de-energize it; insert a short HAL_Delay() between state changes for demonstration.
5. Flash the firmware and confirm an audible relay click accompanies each state change, with the load switching on/off accordingly.
6. Extend the logic so B1 (the user push button, debounced as in Experiment 4) toggles the relay state on each press.

Program:

Viva-voce (In-Lab):

1. Why can a GPIO pin not directly drive a relay coil without a transistor or opto-isolator stage?

2. What is a flyback/freewheeling diode and why is one typically placed across a relay coil?

3. Why are most low-cost relay modules active-LOW rather than active-HIGH?

4. What additional isolation would be required if the switched load were mains AC voltage rather than a low-voltage DC lamp?

Post Lab Requirements:

1. Save the screenshots/pictures of the output obtained and bring the same to the next lab.
2. Note the procedures and observations as instructed by your advisor.
3. Submit the practical report in the next lab for correction.

Results

The relay produces an audible click and the connected load switches on and off in step with the 2-second GPIO toggle interval, confirming correct logic-level isolation between the 3.3 V MCU domain and the higher-power load circuit.

Analysis and Inferences:

Evaluator Remark (if Any):	Marks Secured: _____ out of 50
	Signature of the Evaluator with Date

Evaluator MUST ask Viva-voce prior to signing and posting marks for each experiment.

Experiment 7

Interfacing an I²C 16×2 LCD and Displaying User Information with Relay Status

Aim/Objective: To interface a 16x2 character LCD via an I2C PCF8574 backpack, and to display static user information together with the live status of the relay from Experiment 6.

PRE-REQUISITE:

1. Completion of Experiment 6 (GPIO output driving external hardware).
2. Basic understanding of the I2C bus protocol.

Tools & Components Required:

- STM32CubeIDE, STM32CubeMX & STM32 HAL Library
- ST Nucleo-F446RE (STM32F446RET6, Arm Cortex-M4 @ 180 MHz)
- 16x2 character LCD with PCF8574 I2C backpack (address 0x27 or 0x3F)
- 5 V relay module (from Experiment 6)
- Jumper wires
- Logic-level-safe I2C pull-ups (usually already fitted on the backpack)

Description:

The PCF8574 is an 8-bit I/O expander with an I2C interface; the common LCD 'I2C backpack' uses it to drive the HD44780 LCD controller's 4-bit data bus, RS, RW, and Enable lines, plus the backlight, over just two wires (SDA, SCL) instead of the six-plus pins a parallel HD44780 connection would require.

HD44780-compatible LCDs require a specific initialization sequence (function set, display on/off, entry mode set, clear display) before characters can be written; libraries typically wrap this sequence together with the I2C byte-shifting needed to toggle the Enable line through the PCF8574.

Circuit / Pin Connections:

Signal	Nucleo-F446RE Pin	Notes
LCD SDA	PB9 (I2C1_SDA)	Open-drain, requires pull-up (present on most backpacks)
LCD SCL	PB8 (I2C1_SCL)	Open-drain, requires pull-up
LCD VCC	5V	Backpack logic and backlight supply
LCD GND	GND	Common ground
Relay IN	PB0	As wired in Experiment 6

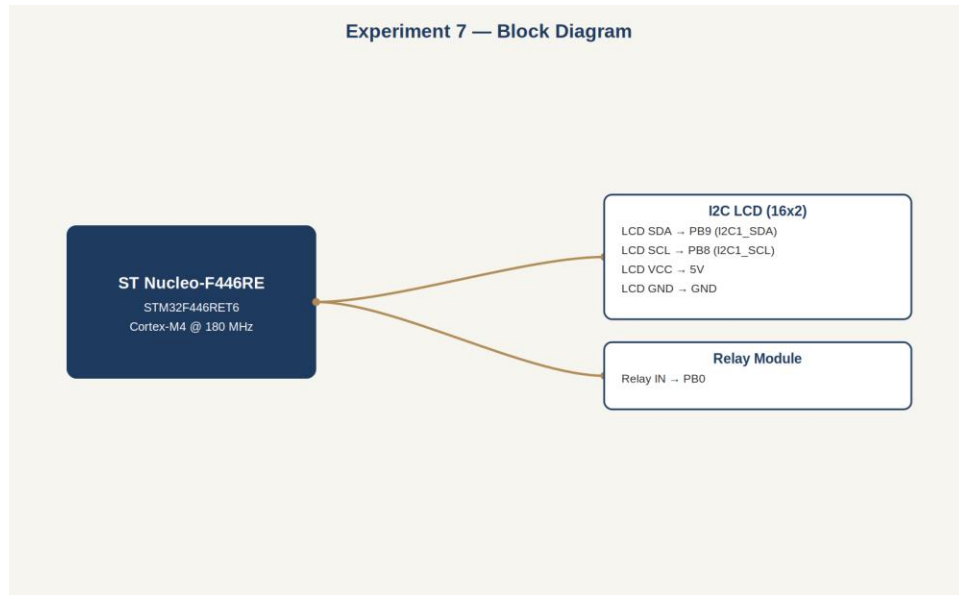


Figure: Block diagram for Experiment 7

Procedure:

Configure the required peripherals in STM32CubeMX and implement a HAL-based program to achieve the objective stated above, following the procedure and pin connections given below. Build, flash, and verify the functionality on the Nucleo-F446RE hardware.

1. Wire the LCD backpack's SDA/SCL to PB9/PB8, and VCC/GND to 5V/GND.
2. In CubeMX, enable I2C1 in the Connectivity panel with standard mode (100 kHz) timing, and keep PB0 configured as GPIO_Output for the relay as in Experiment 6.
3. Add (or write) a small HD44780-over-PCF8574 driver providing `Lcd_Init()`, `Lcd_SetCursor(row, col)`, and `Lcd_Print(char*)` built on `HAL_I2C_Master_Transmit()`.
4. In `main.c`, call `Lcd_Init()` once, then print a static user information line (e.g. student name / roll number) on row 0.
5. On row 1, periodically print 'Relay: ON' or 'Relay: OFF' reflecting the current state of the PB0-driven relay from Experiment 6.
6. Flash and verify the LCD displays both lines correctly, with row 1 updating live as the relay is toggled (e.g. via the debounced push button).

Program:

Viva-voce (In-Lab):

1. How does the PCF8574 reduce the LCD's parallel HD44780 interface down to two wires?

2. What is the purpose of the Enable (E) line pulse in the HD44780 write cycle, and how is it emulated over I2C?

3. How would you determine the correct 7-bit I2C address of an unknown PCF8574 backpack (0x20-0x27 or 0x38-0x3F range)?

4. Why must I2C SDA/SCL lines be open-drain with pull-up resistors rather than push-pull outputs?

Post Lab Requirements:

1. Save the screenshots/pictures of the output obtained and bring the same to the next lab.
2. Note the procedures and observations as instructed by your advisor.
3. Submit the practical report in the next lab for correction.

Results

The LCD's first row shows the fixed user information string persistently, while the second row correctly reflects the live relay state, updating within a fraction of a second of each relay toggle -- confirming both successful I2C communication with the PCF8574 backpack and correct polling of the relay's GPIO state.

Analysis and Inferences:

Evaluator Remark (if Any):	Marks Secured: _____ out of 50
	Signature of the Evaluator with Date

Evaluator MUST ask Viva-voce prior to signing and posting marks for each experiment.

Experiment 8

Developing and Analyzing UART Serial Communication: Data Transmission from STM32 to PC

Aim/Objective: To configure USART2 on the Nucleo-F446RE and transmit data from the STM32 to a PC terminal over the on-board ST-LINK Virtual COM Port (VCP).

PRE-REQUISITE:

1. Completion of Experiment 1 (STM32CubeIDE project workflow).
2. Basic understanding of asynchronous serial communication.

Tools & Components Required:

- STM32CubeIDE, STM32CubeMX & STM32 HAL Library
- ST Nucleo-F446RE (STM32F446RET6, Arm Cortex-M4 @ 180 MHz)
- USB cable (also carries the VCP)
- PC terminal application (e.g. Tera Term, PuTTY, or the Serial Monitor in STM32CubeIDE)

Description:

UART (Universal Asynchronous Receiver/Transmitter) communicates using two independent lines (TX, RX) with no shared clock; both ends must agree in advance on baud rate, word length, parity, and stop bits so each can correctly sample the asynchronous bit stream.

On the Nucleo-F446RE, USART2 (PA2 = TX, PA3 = RX) is routed through the on-board ST-LINK/V2-1, which re-exposes it to the host PC as a standard Virtual COM Port over the same USB cable used for programming -- no external USB-to-serial adapter is required.

Circuit / Pin Connections:

Signal	Nucleo-F446RE Pin	Notes
USART2_TX	PA2	Routed internally to ST-LINK VCP -> USB -> PC
USART2_RX	PA3	Routed internally to ST-LINK VCP

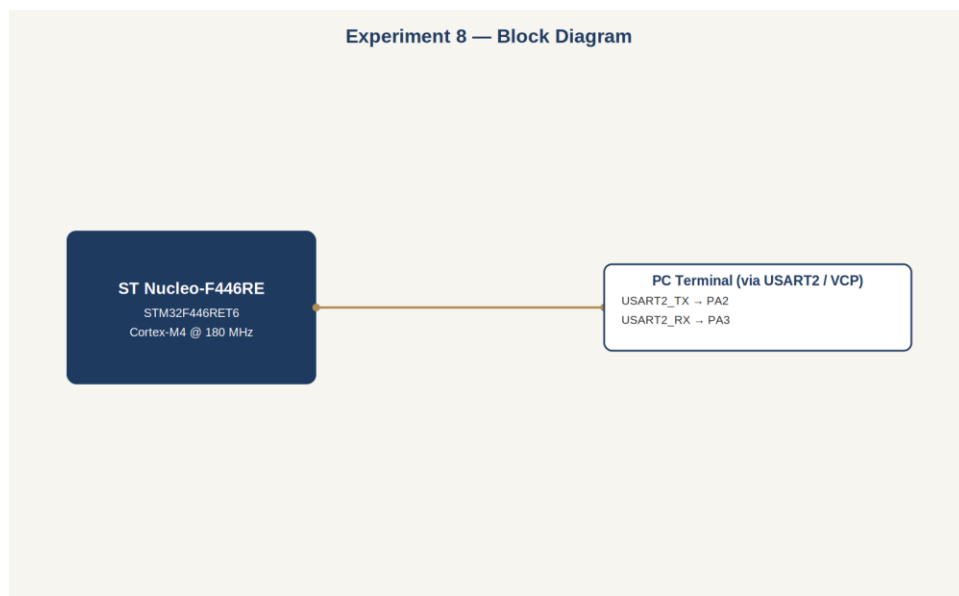


Figure: Block diagram for Experiment 8

Procedure:

Configure the required peripherals in STM32CubeMX and implement a HAL-based program to achieve the objective stated above, following the procedure and pin connections given below. Build, flash, and verify the functionality on the Nucleo-F446RE hardware.

1. In CubeMX, enable USART2 in Asynchronous mode; set Baud Rate = 115200, Word Length = 8 bits, Parity = None, Stop Bits = 1.
2. Generate code; note the auto-generated MX_USART2_UART_Init() and the global handle huart2.
3. On the PC, open a terminal application, select the ST-LINK VCP's COM port, and set it to the same 115200-8-N-1 configuration.
4. In main.c, use HAL_UART_Transmit(&huart2, (uint8_t*)msg, strlen(msg), HAL_MAX_DELAY) to send a greeting string once at start-up.
5. Inside while(1), transmit a periodic message (e.g. an incrementing counter) every second using HAL_Delay(1000).
6. Flash the firmware and confirm the PC terminal displays the greeting followed by the periodic counter messages.

Program:

Viva-voce (In-Lab):

1. Why must both the STM32 and the PC terminal be configured to the identical baud rate, word length, parity, and stop-bit settings?

2. What happens electrically on the USART2 lines during a start bit and a stop bit?

3. What is the practical difference between HAL_UART_Transmit() (polling) and its interrupt/DMA-based counterparts?

4. How is the ST-LINK able to present a VCP over the same USB cable used for SWD programming?

Post Lab Requirements:

1. Save the screenshots/pictures of the output obtained and bring the same to the next lab.
2. Note the procedures and observations as instructed by your advisor.
3. Submit the practical report in the next lab for correction.

Results

The PC terminal displays the initial greeting message followed by an incrementing 'Counter = N' line once per second, confirming correct USART2 configuration, baud-rate agreement, and successful use of the ST-LINK Virtual COM Port bridge.

Analysis and Inferences:

Evaluator Remark (if Any):	Marks Secured: _____ out of 50
	Signature of the Evaluator with Date

Evaluator MUST ask Viva-voce prior to signing and posting marks for each experiment.

Experiment 9

Developing and Analyzing UART Serial Communication: Data Reception from PC to STM32

Aim/Objective: To receive data sent from a PC terminal into the STM32 over USART2 using polling-mode HAL_UART_Receive(), and to act on the received byte(s).

PRE-REQUISITE:

1. Completion of Experiment 8 (UART transmission from STM32 to PC).
2. Basic understanding of receive buffers in serial communication.

Tools & Components Required:

- STM32CubeIDE, STM32CubeMX & STM32 HAL Library
- ST Nucleo-F446RE (STM32F446RET6, Arm Cortex-M4 @ 180 MHz)
- USB cable
- PC terminal application capable of sending keystrokes (e.g. Tera Term, PuTTY)

Description:

Receiving data is the mirror operation of Experiment 8: the STM32 samples the USART2_RX line at the agreed baud rate and reconstructs each byte in the USART peripheral's receive shift register, which HAL_UART_Receive() copies into a user buffer once the requested number of bytes has arrived (or a timeout expires).

In polling mode, HAL_UART_Receive() blocks the CPU until the requested byte count is received or the timeout elapses, which is acceptable for simple command-response demonstrations but not for applications that must remain responsive while waiting for input (addressed later using interrupts in Experiment 11).

Circuit / Pin Connections:

Signal	Nucleo-F446RE Pin	Notes
USART2_RX	PA3	Receives bytes typed in the PC terminal
USART2_TX	PA2	Used to echo/acknowledge what was received
LD2 (on-board LED)	PA5	Toggled in response to a received command character

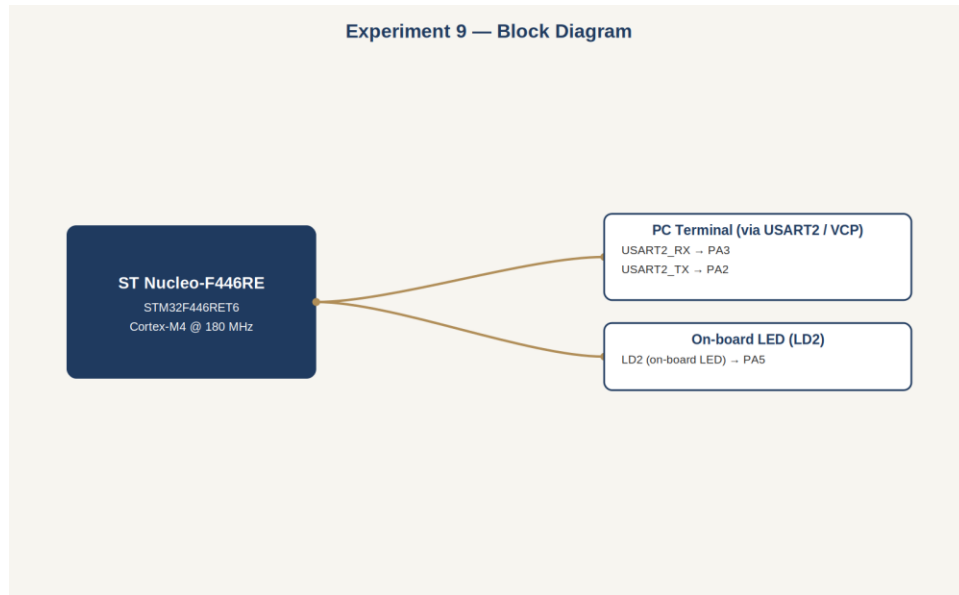


Figure: Block diagram for Experiment 9

Procedure:

Configure the required peripherals in STM32CubeMX and implement a HAL-based program to achieve the objective stated above, following the procedure and pin connections given below. Build, flash, and verify the functionality on the Nucleo-F446RE hardware.

1. Configure USART2 exactly as in Experiment 8 (115200-8-N-1).
2. In main.c, declare a 1-byte receive buffer, e.g. `uint8_t rxByte;`.
3. Inside `while(1)`, call `HAL_UART_Receive(&huart2, &rxByte, 1, HAL_MAX_DELAY)` to block until a character arrives.
4. If the received character is '1', turn LD2 ON; if it is '0', turn LD2 OFF; otherwise transmit an 'Unknown command' message.
5. Flash the firmware, open the PC terminal, and type '1' then Enter, then '0' then Enter, observing LD2 respond accordingly.
6. Send an unrecognized character (e.g. 'x') and confirm the STM32 replies with the error message.

Program:

Viva-voce (In-Lab):

1. What determines how long HAL_UART_Receive() blocks when called with HAL_MAX_DELAY?

2. What happens if the PC sends bytes faster than the firmware calls HAL_UART_Receive() again (in polling mode)?

3. How would you extend this single-byte command scheme to accept multi-character commands terminated by '\r\n'?

4. What USART status flag indicates a received data register is ready to be read, and which HAL function polls it internally?

Post Lab Requirements:

1. Save the screenshots/pictures of the output obtained and bring the same to the next lab.
2. Note the procedures and observations as instructed by your advisor.
3. Submit the practical report in the next lab for correction.

Results

Sending '1' from the PC terminal turns LD2 on and the STM32 replies 'LED ON'; sending '0' turns it off with an 'LED OFF' reply; any other character produces an 'Unknown command' response -- confirming correct byte-level UART reception and command decoding.

Analysis and Inferences:

Evaluator Remark (if Any):	Marks Secured: _____ out of 50
	Signature of the Evaluator with Date

Evaluator MUST ask Viva-voce prior to signing and posting marks for each experiment.

Experiment 10

Development of a UART Echo Application Using STM32 Displaying on LCD

Aim/Objective: To build a UART echo application that reflects every character typed on the PC terminal back to the PC, while also displaying the most recently received line on the I2C LCD from Experiment 7.

PRE-REQUISITE:

1. Completion of Experiments 7 and 9 (I2C LCD interfacing and UART reception).
2. Basic understanding of string buffering.

Tools & Components Required:

- STM32CubeIDE, STM32CubeMX & STM32 HAL Library
- ST Nucleo-F446RE (STM32F446RET6, Arm Cortex-M4 @ 180 MHz)
- PC terminal application
- 16x2 I2C LCD (from Experiment 7)

Description:

An echo application is the classic way to verify a serial link is fully bidirectional and byte-accurate: every byte received on RX is immediately re-transmitted on TX, so the sender sees their own keystrokes reflected back.

This experiment combines Experiments 7 and 9: received characters are both echoed back over UART and accumulated into a line buffer that is rendered to the LCD once a carriage return is detected, demonstrating multi-peripheral integration (UART + I2C) in a single application.

Circuit / Pin Connections:

Signal	Nucleo-F446RE Pin	Notes
USART2 TX/RX	PA2 / PA3	PC terminal link, 115200-8-N-1
LCD SDA/SCL	PB9 / PB8	As wired in Experiment 7

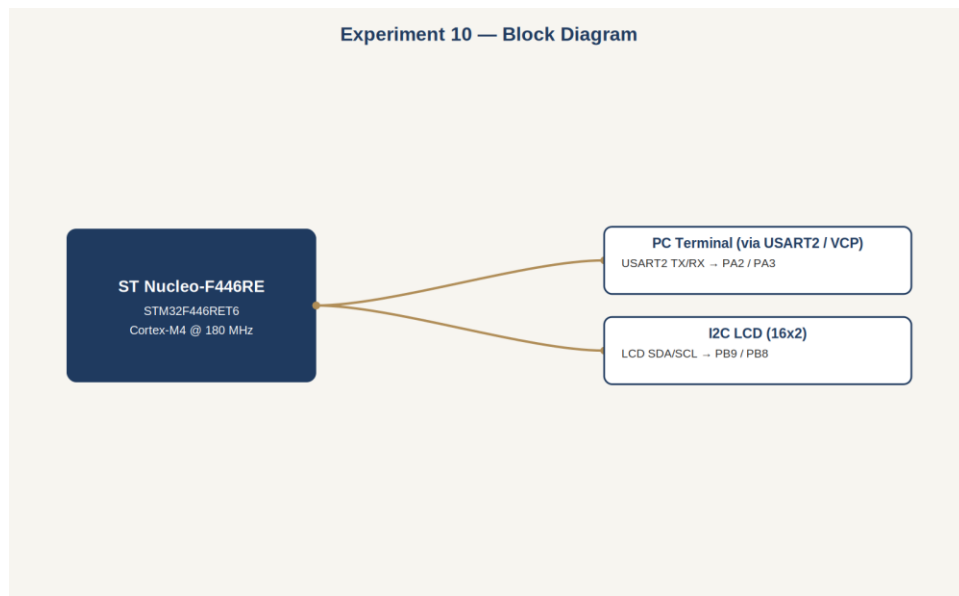


Figure: Block diagram for Experiment 10

Procedure:

Configure the required peripherals in STM32CubeMX and implement a HAL-based program to achieve the objective stated above, following the procedure and pin connections given below. Build, flash, and verify the functionality on the Nucleo-F446RE hardware.

1. Configure USART2 (Experiment 8/9) and I2C1 + the LCD driver (Experiment 7) together in the same project.
2. Declare a line buffer char line[17] and an index lineIdx = 0.
3. In the receive loop, echo each received byte immediately with HAL_UART_Transmit().
4. If the byte is not '\r', append it to line[lineIdx++] (bounded to 16 characters, the LCD row width).
5. If the byte is '\r' (Enter), null-terminate line[], display it on LCD row 1 via Lcd_SetCursor(1,0) and Lcd_Print(line), then reset lineIdx = 0.
6. Flash, open the PC terminal, type a short phrase, and confirm it is echoed character-by-character and then displayed in full on the LCD after pressing Enter.

Program:

Viva-voce (In-Lab):

1. Why is it necessary to null-terminate the line[] buffer before passing it to Lcd_Print()?

2. What would happen if the user typed more than 16 characters before pressing Enter, and how does the code guard against it?

3. Why is echoing performed one byte at a time rather than buffering the whole line before echoing?

4. How could this design be adapted to also echo to the LCD in real time as each character arrives, rather than only on Enter?

Post Lab Requirements:

1. Save the screenshots/pictures of the output obtained and bring the same to the next lab.
2. Note the procedures and observations as instructed by your advisor.
3. Submit the practical report in the next lab for correction.

Results

Every keystroke sent from the PC terminal is echoed back immediately, confirming a working full-duplex link, and pressing Enter causes the complete typed line to appear on the LCD's second row -- demonstrating correct coordination between the UART receive path and the I2C LCD driver.

Analysis and Inferences:

Evaluator Remark (if Any):	Marks Secured: _____ out of 50
	Signature of the Evaluator with Date

Evaluator MUST ask Viva-voce prior to signing and posting marks for each experiment.

Experiment 11

Designing Asynchronous Data Interrupt-Driven UART Communication Using HAL Library

Aim/Objective: To replace polling-mode UART reception with interrupt-driven reception using HAL_UART_Receive_IT(), freeing the CPU to perform other work while waiting for incoming serial data.

PRE-REQUISITE:

1. Completion of Experiment 9 (polling-based UART reception).
2. Basic understanding of interrupt service routines (ISRs).

Tools & Components Required:

- STM32CubeIDE, STM32CubeMX & STM32 HAL Library
- ST Nucleo-F446RE (STM32F446RET6, Arm Cortex-M4 @ 180 MHz)
- PC terminal application

Description:

Polling-mode HAL_UART_Receive() blocks the CPU for the entire wait, which does not scale once an application must service several peripherals concurrently. The STM32 USART peripheral can instead raise an interrupt (RXNE -- Receive Not Empty) as soon as a byte arrives; the NVIC then vectors to the USART2_IRQHandler, which HAL routes to HAL_UART_RxCpltCallback() once the requested byte count has been received.

HAL_UART_Receive_IT() arms a single interrupt-driven reception of a given length and returns immediately; the application must re-arm it (call it again) inside the completion callback to continue receiving further bytes -- a pattern common to all HAL '_IT' and '_DMA' peripheral APIs.

Circuit / Pin Connections:

Signal	Nucleo-F446RE Pin	Notes
USART2 TX/RX	PA2 / PA3	115200-8-N-1, NVIC USART2 global interrupt enabled
LD2 (on-board LED)	PA5	Toggled from inside the RX-complete callback

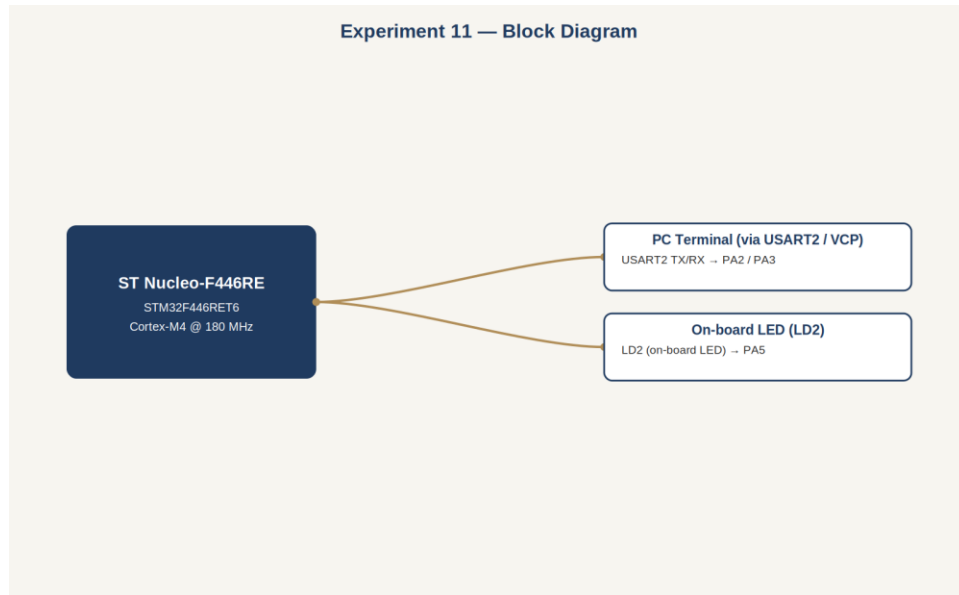


Figure: Block diagram for Experiment 11

Procedure:

Configure the required peripherals in STM32CubeMX and implement a HAL-based program to achieve the objective stated above, following the procedure and pin connections given below. Build, flash, and verify the functionality on the Nucleo-F446RE hardware.

1. Configure USART2 as before; in CubeMX's NVIC settings tab, enable the 'USART2 global interrupt'.
2. In main.c, declare a global `uint8_t rxByte` and call `HAL_UART_Receive_IT(&huart2, &rxByte, 1)` once before entering `while(1)`.
3. Implement `void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)` in main.c: check if (`huart->Instance == USART2`), toggle LD2, and immediately re-arm reception with another `HAL_UART_Receive_IT()` call.
4. Leave the `while(1)` loop otherwise empty (or executing an unrelated task) to demonstrate the CPU is free while awaiting UART data.
5. Flash the firmware and send characters from the PC terminal; confirm LD2 toggles once per received character with no blocking delay in the main loop.
6. Add instrumentation (e.g. a free-running counter incremented in the main loop) to prove the main loop keeps running between UART interrupts.

Program:

Viva-voce (In-Lab):

1. What NVIC configuration step is required in CubeMX before HAL_UART_Receive_IT() interrupts will actually fire?

2. Why must HAL_UART_Receive_IT() be called again inside its own completion callback to keep receiving further bytes?

3. What would happen if a second byte arrived before the callback finished re-arming reception -- how does the USART overrun (ORE) flag relate to this?

4. How does HAL_UART_Receive_IT() differ operationally from HAL_UART_Receive_DMA()?

Post Lab Requirements:

1. Save the screenshots/pictures of the output obtained and bring the same to the next lab.
2. Note the procedures and observations as instructed by your advisor.
3. Submit the practical report in the next lab for correction.

Results

LD2 toggles reliably once for each character typed in the PC terminal, with essentially zero latency, and the idleLoops counter is observed (via debugger watch) to increase continuously between characters -- confirming the CPU is never blocked waiting for UART data, unlike the polling approach of Experiment 9.

Analysis and Inferences:

Evaluator Remark (if Any):	Marks Secured: _____ out of 50
	Signature of the Evaluator with Date

Evaluator MUST ask Viva-voce prior to signing and posting marks for each experiment.

Experiment 12

Synthesizing Data Streams from UART to I2C Peripheral Displays

Aim/Objective: To parse structured commands arriving over interrupt-driven UART and route the decoded values to the I2C LCD, synthesizing a UART-to-I2C data bridge.

PRE-REQUISITE:

1. Completion of Experiments 7 and 11 (I2C LCD display and interrupt-driven UART).
2. Basic understanding of data flow between two different peripheral interfaces.

Tools & Components Required:

- STM32CubeIDE, STM32CubeMX & STM32 HAL Library
- ST Nucleo-F446RE (STM32F446RET6, Arm Cortex-M4 @ 180 MHz)
- PC terminal application
- 16x2 I2C LCD (Experiment 7)

Description:

Many embedded systems must translate a byte stream arriving on one interface into structured updates on another. Here, a simple line-based protocol (ASCII text terminated by '\n') sent over UART is parsed and the extracted fields are written to specific rows/columns of the I2C LCD, illustrating a general UART-to-I2C data synthesis pattern applicable to more complex sensor/actuator bridges later in the course.

Building on the interrupt-driven reception of Experiment 11, each received byte is appended to a ring/line buffer inside the ISR callback; the main loop polls a 'line ready' flag set once '\n' is seen, and only then performs the (slower) I2C transaction outside interrupt context -- keeping the ISR itself short.

Circuit / Pin Connections:

Signal	Nucleo-F446RE Pin	Notes
USART2 TX/RX	PA2 / PA3	Interrupt-driven, as in Experiment 11
LCD SDA/SCL	PB9 / PB8	As wired in Experiment 7

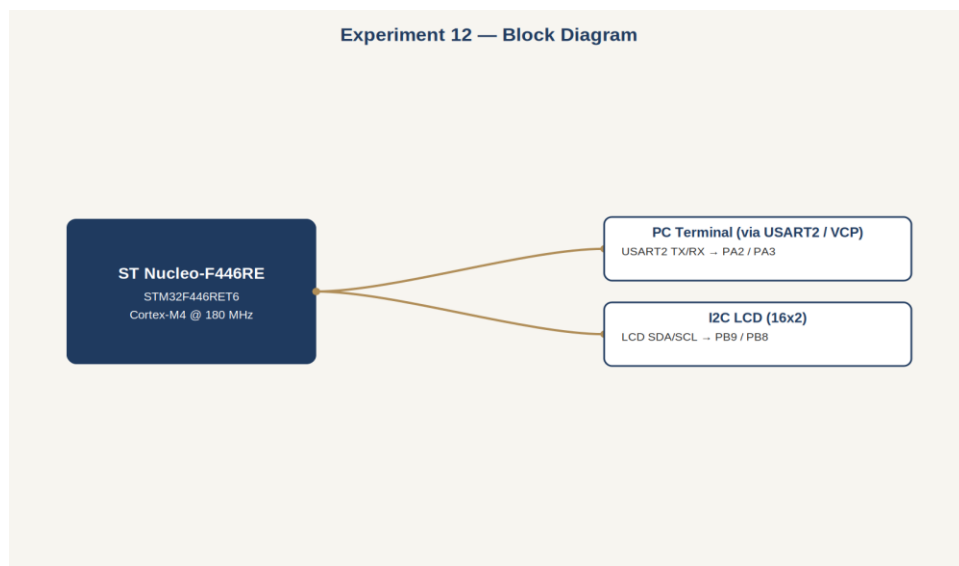


Figure: Block diagram for Experiment 12

Procedure:

Configure the required peripherals in STM32CubeMX and implement a HAL-based program to achieve the objective stated above, following the procedure and pin connections given below. Build, flash, and verify the functionality on the Nucleo-F446RE hardware.

1. Combine the interrupt-driven UART reception of Experiment 11 with the LCD driver of Experiment 7 in one project.
2. In HAL_UART_RxCpltCallback(), append the received byte to a global line buffer instead of toggling an LED; set a volatile flag lineReady = 1 when '\n' is received.
3. Define a simple protocol, e.g. 'L1:Hello World\n' to write to LCD row 0, and 'L2:...' for row 1.
4. In the main loop, poll if (lineReady) { parse the 'L1:/'L2:' prefix, update the corresponding LCD row, clear the buffer, lineReady = 0; }.
5. Flash the firmware and, from the PC terminal, send 'L1:Lab 12 Demo\n' followed by 'L2:UART to I2C\n', confirming both rows update as commanded.
6. Send a malformed line without a recognized prefix and confirm it is safely ignored (no crash, no corrupted display).

Program:

Viva-voce (In-Lab):

1. Why is the I2C transaction performed in the main loop rather than directly inside HAL_UART_RxCpltCallback()?

2. What risk would arise if lineBuf were written to from the ISR while simultaneously being read by the main loop without a 'ready' flag?

3. How would you extend the protocol to support a checksum or start-of-frame marker for robustness against corrupted transmissions?

4. Why is strncmp() used instead of strcmp() to check the 'L1:/'L2:' prefix?

Post Lab Requirements:

1. Save the screenshots/pictures of the output obtained and bring the same to the next lab.
2. Note the procedures and observations as instructed by your advisor.
3. Submit the practical report in the next lab for correction.

Results

Sending 'L1:...' and 'L2:...' lines from the PC terminal correctly updates the corresponding LCD row within the main loop's next iteration, and malformed lines are safely discarded, confirming a working, robust UART-to-I2C data synthesis bridge.

Analysis and Inferences:

Evaluator Remark (if Any):	Marks Secured: _____ out of 50
	Signature of the Evaluator with Date

Evaluator MUST ask Viva-voce prior to signing and posting marks for each experiment.

Experiment 13

Developing a Rule-Based Automated Control System Using UART Communication

Aim/Objective: To implement a simple rule-based control system in which UART commands received from a PC set operating parameters, and the STM32 autonomously applies IF-THEN rules to drive the relay and LED outputs.

PRE-REQUISITE:

1. Completion of Experiments 6 and 11 (relay/output control and interrupt-driven UART).
2. Basic understanding of conditional (if-else) control logic.

Tools & Components Required:

- STM32CubeIDE, STM32CubeMX & STM32 HAL Library
- ST Nucleo-F446RE (STM32F446RET6, Arm Cortex-M4 @ 180 MHz)
- PC terminal application
- 5 V relay module (Experiment 6)

Description:

A rule-based (or 'expert system style') controller evaluates a small set of IF-condition-THEN-action rules against its current state on every control cycle, rather than reacting to a single command in isolation. This experiment uses UART-received parameters (a threshold and a mode flag) as the condition inputs and drives the relay/LED as the actions, previewing the sensor-driven automated control experiments later in the course (Experiments 22-23).

Because the rules are re-evaluated continuously in the main loop rather than only at the moment a command arrives, the system responds automatically and consistently to state changes, which is the defining characteristic of rule-based automation versus simple command-response.

Circuit / Pin Connections:

Signal	Nucleo-F446RE Pin	Notes
USART2 TX/RX	PA2 / PA3	Interrupt-driven command input, as in Experiment 11
Relay IN	PB0	Actuator output, as wired in Experiment 6
LD2	PA5	Status indicator

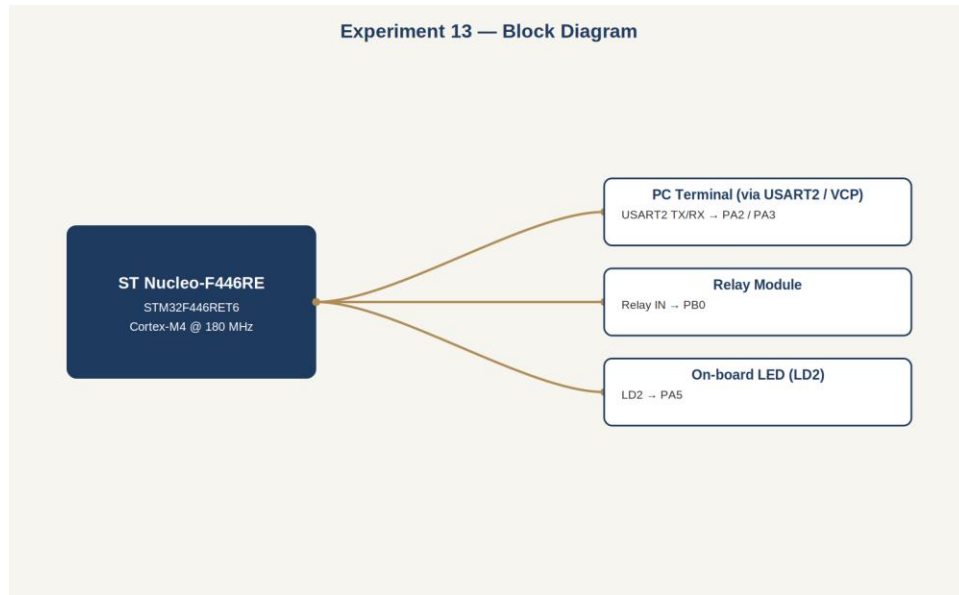


Figure: Block diagram for Experiment 13

Procedure:

Configure the required peripherals in STM32CubeMX and implement a HAL-based program to achieve the objective stated above, following the procedure and pin connections given below. Build, flash, and verify the functionality on the Nucleo-F446RE hardware.

1. Combine interrupt-driven UART (Experiment 11) with the relay output (Experiment 6) in one project.
2. Define a simple command protocol: 'MODE:AUTO\n', 'MODE:MANUAL\n', 'SET:<value>\n', and 'RELAY:ON\n' / 'RELAY:OFF\n' (manual override).
3. Maintain global state variables: enum {AUTO, MANUAL} mode; and int threshold; and a simulated 'level' variable (or wire in a real potentiometer via ADC from Experiment 18 as an extension).
4. Parse incoming lines (reusing the line-buffer technique of Experiment 12) to update mode, threshold, or force the relay state directly under MANUAL mode.
5. On every main-loop iteration, apply the rule: IF mode == AUTO AND level > threshold THEN energize relay ELSE de-energize relay.
6. Flash the firmware; in AUTO mode, vary 'level' (e.g. via a debug variable or an ADC input) and confirm the relay follows the rule automatically; switch to MANUAL and confirm RELAY:ON/OFF commands now take direct control.

Program:

Viva-voce (In-Lab):

1. What distinguishes a 'rule-based' controller from a simple command-response interface?

2. Why is ApplyRules() called every main-loop iteration rather than only when a new UART command arrives?

3. How would you add hysteresis (separate ON/OFF thresholds) to prevent relay chatter when 'level' hovers near the threshold?

4. What safety consideration applies when switching from AUTO back to MANUAL mode regarding the relay's last commanded state?

Post Lab Requirements:

1. Save the screenshots/pictures of the output obtained and bring the same to the next lab.
2. Note the procedures and observations as instructed by your advisor.
3. Submit the practical report in the next lab for correction.

Results

In AUTO mode the relay energizes and de-energizes automatically as 'level' crosses the configured threshold, with no further UART commands required; sending 'MODE:MANUAL' followed by RELAY:ON/OFF immediately overrides the automatic rule, demonstrating a working rule-based control loop layered on top of the UART command interface.

Analysis and Inferences:

Evaluator Remark (if Any):	Marks Secured: _____ out of 50
	Signature of the Evaluator with Date

Evaluator MUST ask Viva-voce prior to signing and posting marks for each experiment.

Experiment 14

Designing a PWM Signal Generation Using STM32 Timers

Aim/Objective: To configure a general-purpose timer in PWM output mode and generate a variable duty-cycle PWM signal on a GPIO pin, verified on the on-board LED and an oscilloscope.

PRE-REQUISITE:

1. Completion of Experiment 1 (project setup) and basic timer concepts.
2. Basic understanding of duty cycle and frequency.

Tools & Components Required:

- STM32CubeIDE, STM32CubeMX & STM32 HAL Library
- ST Nucleo-F446RE (STM32F446RET6, Arm Cortex-M4 @ 180 MHz)
- Oscilloscope or logic analyzer (to observe duty cycle)

Description:

STM32 general-purpose timers (e.g. TIM3) can generate PWM directly in hardware: a counter (CNT) counts up to a programmable Auto-Reload value (ARR), and a Capture/Compare register (CCR) sets the point within that count at which the output pin switches state, giving a duty cycle of CCR/ARR without any CPU intervention once configured.

PWM frequency is set by the timer's input clock divided by the Prescaler (PSC) and the Auto-Reload value: $f_PWM = f_TIMCLK / ((PSC+1) \times (ARR+1))$. Duty cycle is then independently adjustable in real time simply by writing a new value to CCR.

Circuit / Pin Connections:

Signal	Nucleo-F446RE Pin	Notes
TIM3_CH1 (PWM output)	PA6	Alternate function AF2; probe here with the oscilloscope

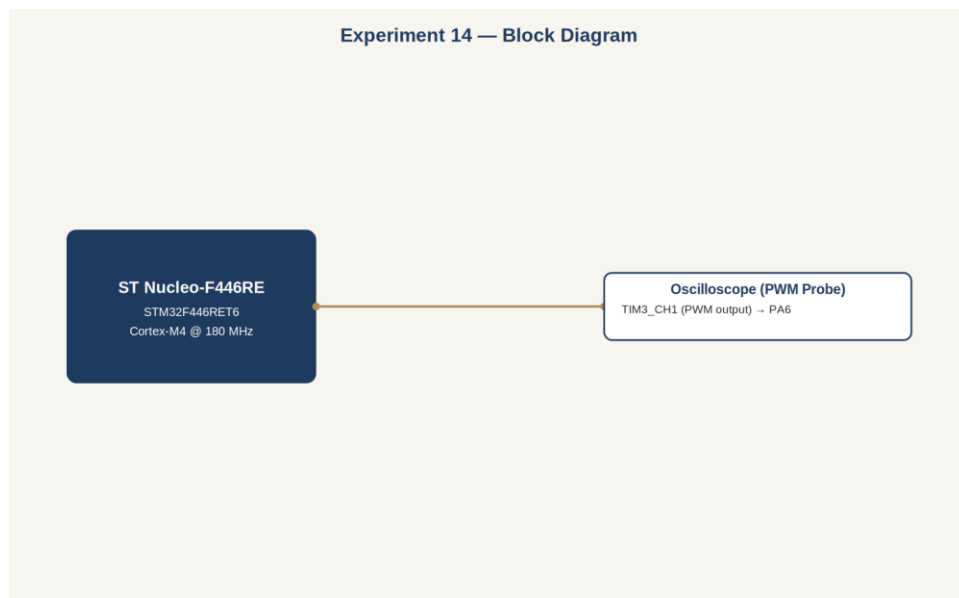


Figure: Block diagram for Experiment 14

Procedure:

Configure the required peripherals in STM32CubeMX and implement a HAL-based program to achieve the objective stated above, following the procedure and pin connections given below. Build, flash, and verify the functionality on the Nucleo-F446RE hardware.

1. In CubeMX, enable TIM3 and set Channel 1 to 'PWM Generation CH1'.
2. Set Prescaler and Counter Period (ARR) to produce a convenient frequency, e.g. PSC = 89, ARR = 999 for a 1 kHz PWM at 90 MHz APB1 timer clock ($90\text{e}6 / (90 \times 1000) = 1 \text{ kHz}$).
3. Generate code; in main.c call `HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_1)` to enable the PWM output.
4. Sweep the duty cycle by writing `__HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, dutyValue)` with dutyValue ranging from 0 to ARR inside a slow loop (e.g. ramping up and down every 20 ms).
5. Probe PA6 with an oscilloscope and confirm the measured frequency matches the calculated 1 kHz, and that the duty cycle visibly ramps from 0% to 100% and back.
6. Note the average voltage on the oscilloscope's DC-coupled measurement and confirm it tracks the duty-cycle percentage (e.g. ~50% duty gives ~1.65 V average for a 3.3 V logic swing).

Program:

Viva-voce (In-Lab):

1. Derive the PWM frequency formula in terms of timer clock, prescaler, and auto-reload value, and verify it against the 1 kHz example.

2. What is the maximum achievable duty-cycle resolution given $ARR = 999$?

3. What is the difference between PWM Mode 1 and PWM Mode 2 on an STM32 timer output compare channel?

4. Why does increasing ARR to get finer duty-cycle resolution reduce the achievable PWM frequency for a fixed timer clock?

Post Lab Requirements:

1. Save the screenshots/pictures of the output obtained and bring the same to the next lab.
2. Note the procedures and observations as instructed by your advisor.
3. Submit the practical report in the next lab for correction.

Results

The oscilloscope shows a stable 1 kHz square wave on PA6 whose duty cycle continuously ramps between 0% and 100%, matching the calculated timer frequency and confirming correct PWM channel configuration and real-time compare-register updates.

Analysis and Inferences:

Evaluator Remark (if Any):	Marks Secured: _____ out of 50
	Signature of the Evaluator with Date

Evaluator MUST ask Viva-voce prior to signing and posting marks for each experiment.

Experiment 15

Designing a Servo Motor Position Control Using PWM

Aim/Objective: To control the angular position of an SG90-class hobby servo motor using the TIM3 PWM output configured in Experiment 14, by varying pulse width within a 20 ms frame.

PRE-REQUISITE:

1. Completion of Experiment 14 (PWM signal generation).
2. Basic understanding of hobby servo motor control.

Tools & Components Required:

- STM32CubeIDE, STM32CubeMX & STM32 HAL Library
- ST Nucleo-F446RE (STM32F446RET6, Arm Cortex-M4 @ 180 MHz)
- SG90 (or similar) micro servo motor
- External 5 V supply for the servo (recommended over the Nucleo 5V rail for adequate current)
- Jumper wires

Description:

Hobby servos are controlled by a pulse-position modulation scheme: a control pulse is sent once every 20 ms (50 Hz), and the pulse width -- typically 1.0 ms to 2.0 ms -- linearly maps to the servo's angular range (commonly 0 deg to 180 deg), with 1.5 ms corresponding to the center position.

This is the same hardware PWM peripheral as Experiment 14, simply reconfigured for a 50 Hz frame (ARR corresponding to 20 ms) with the compare value expressed directly as a pulse width in the 1-2 ms range rather than a percentage duty cycle.

Circuit / Pin Connections:

Signal	Nucleo-F446RE Pin	Notes
Servo signal (orange/yellow)	PA6 (TIM3_CH1)	PWM control pulse
Servo VCC (red)	External 5V	Do not power directly from the Nucleo 3.3V rail
Servo GND (brown/black)	GND	Common ground with the Nucleo -- required even with an external supply

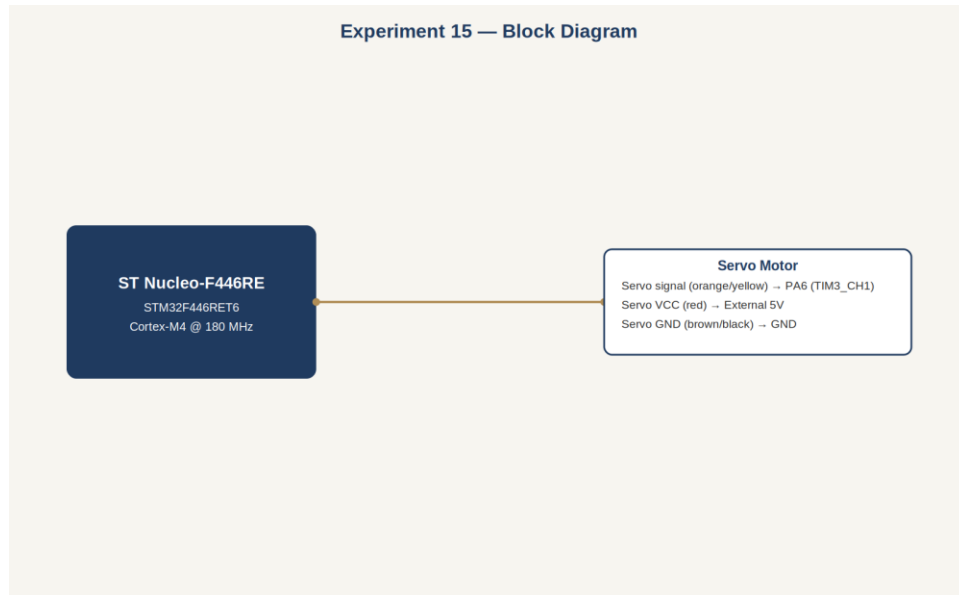


Figure: Block diagram for Experiment 15

Procedure:

Configure the required peripherals in STM32CubeMX and implement a HAL-based program to achieve the objective stated above, following the procedure and pin connections given below. Build, flash, and verify the functionality on the Nucleo-F446RE hardware.

1. Reconfigure TIM3 for a 50 Hz frame: with a 90 MHz timer clock, choose $PSC = 89$ and $ARR = 19999$ so that $90e6/(90 \times 20000) = 50$ Hz (20 ms period), giving 1 μs per timer tick.
2. Wire the servo signal line to PA6, VCC to an external 5V supply, and GND common with the Nucleo.
3. In main.c, define helper macros so that a compare value of 1000 = 1.0 ms (0 deg), 1500 = 1.5 ms (90 deg), and 2000 = 2.0 ms (180 deg), since each timer tick is 1 μs .
4. Call `HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_1)` and sweep `__HAL_TIM_SET_COMPARE()` through 1000 -> 1500 -> 2000 with `HAL_Delay(1000)` between steps to visibly move the servo to 0 deg, 90 deg, and 180 deg.
5. Verify the servo reaches each commanded position smoothly and holds it without jitter.
6. Optionally combine with Experiment 9/11's UART reception so an operator can type an angle to move the servo interactively.

Program:

Viva-voce (In-Lab):

1. Why do hobby servos require a 50 Hz (20 ms) frame rather than the 1 kHz frame used for the LED PWM in Experiment 14?

2. What would happen to the servo's behaviour if the pulse width were driven outside the 1-2 ms range?

3. Why is it recommended to power the servo from an external 5 V supply rather than the Nucleo's own 5 V or 3.3 V rail?

4. How would you convert a desired angle in degrees (0-180) directly into the correct CCR value in code?

Post Lab Requirements:

1. Save the screenshots/pictures of the output obtained and bring the same to the next lab.
2. Note the procedures and observations as instructed by your advisor.
3. Submit the practical report in the next lab for correction.

Results

The servo horn moves smoothly to 0 deg, 90 deg, and 180 deg as each compare value is applied, holding each position steadily without visible jitter, confirming correct 50 Hz frame timing and pulse-width-to-angle mapping.

Analysis and Inferences:

Evaluator Remark (if Any):	Marks Secured: _____ out of 50
	Signature of the Evaluator with Date

Evaluator MUST ask Viva-voce prior to signing and posting marks for each experiment.

Experiment 16

Design and Implementation of a Bidirectional PWM-Controlled DC Motor Drive Using an H-Bridge Driver

Aim/Objective: To drive a brushed DC motor bidirectionally, with variable speed, using an H-bridge driver module (e.g. L298N) controlled by STM32 GPIO direction pins and a TIM3 PWM speed signal.

PRE-REQUISITE:

1. Completion of Experiment 14 (PWM generation).
2. Basic understanding of H-bridge motor driver circuits.

Tools & Components Required:

- STM32CubeIDE, STM32CubeMX & STM32 HAL Library
- ST Nucleo-F446RE (STM32F446RET6, Arm Cortex-M4 @ 180 MHz)
- L298N (or similar) H-bridge motor driver module
- Brushed DC motor
- External motor supply (6-12 V, matched to the motor)
- Jumper wires

Description:

An H-bridge arranges four switches around a motor so that current can flow through it in either direction (forward or reverse) under logic control, while a separate PWM input to the driver's enable pin controls the effective average voltage applied, and therefore the motor's speed, via fast switching.

On the L298N, two digital inputs (IN1, IN2) set the direction (one HIGH/one LOW for forward, reversed for reverse, both LOW/HIGH for brake/stop), while the ENA pin accepts a PWM signal whose duty cycle sets the speed; the module's own power stage isolates the STM32's 3.3 V logic from the motor's higher-current supply.

Circuit / Pin Connections:

Signal	Nucleo-F446RE Pin	Notes
IN1 (direction)	PB1	GPIO output, digital direction control
IN2 (direction)	PB2	GPIO output, digital direction control
ENA (speed / PWM)	PA6 (TIM3_CH1)	PWM duty cycle sets motor speed
Motor supply / GND	External 6-12V supply, common GND	Powers the motor via the L298N's output stage

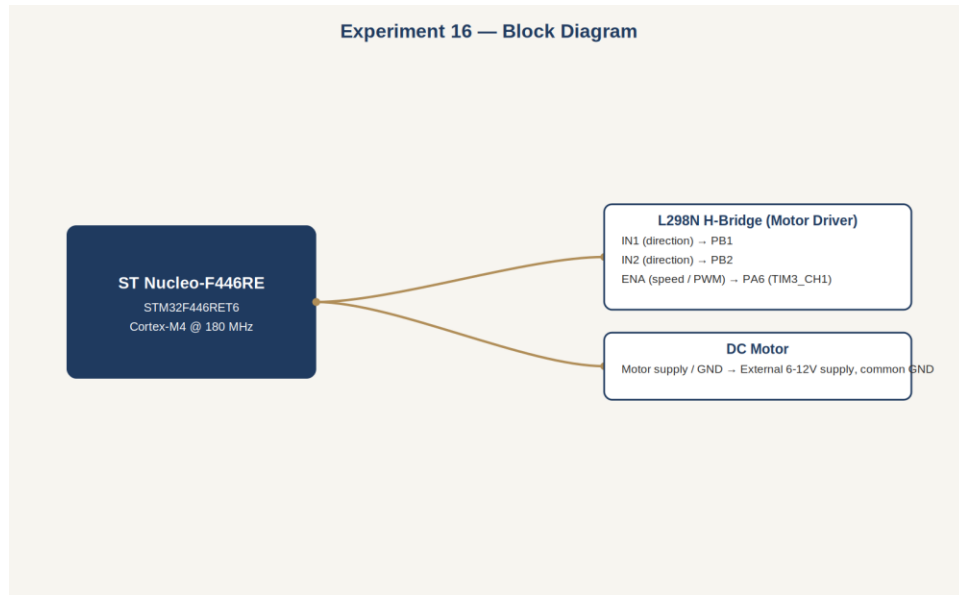


Figure: Block diagram for Experiment 16

Procedure:

Configure the required peripherals in STM32CubeMX and implement a HAL-based program to achieve the objective stated above, following the procedure and pin connections given below. Build, flash, and verify the functionality on the Nucleo-F446RE hardware.

1. Wire IN1/IN2 to PB1/PB2, ENA to PA6 (TIM3_CH1 from Experiment 14), the motor to OUT1/OUT2, and the external motor supply plus common ground to the L298N.
2. In CubeMX, configure PB1 and PB2 as GPIO_Output, and keep TIM3_CH1 configured for a convenient PWM frequency (e.g. 1 kHz, as in Experiment 14).
3. Implement a Motor_Forward(), Motor_Reverse(), and Motor_Stop() helper that set IN1/IN2 accordingly, plus a Motor_SetSpeed(uint16_t duty) that writes __HAL_TIM_SET_COMPARE().
4. In main.c, drive the motor forward at 50% duty for 3 seconds, stop for 1 second, reverse at 50% duty for 3 seconds, then stop.
5. Flash and verify the motor spins in the correct direction for each phase and stops cleanly between direction changes (never switching direction without passing through Stop, to avoid shoot-through current spikes).
6. Vary the PWM duty cycle at a fixed direction and confirm motor speed changes proportionally.

Program:

Viva-voce (In-Lab):

1. Why must the motor always pass through the Stop state (IN1=IN2=LOW) when changing direction, rather than switching directly from forward to reverse?

2. What is 'shoot-through current' in an H-bridge, and why is it dangerous?

3. Why is a separate, higher-current external supply used for the motor rather than powering it from the Nucleo board?

4. How does PWM duty cycle at ENA translate into an effective average voltage seen by the motor?

Post Lab Requirements:

1. Save the screenshots/pictures of the output obtained and bring the same to the next lab.
2. Note the procedures and observations as instructed by your advisor.
3. Submit the practical report in the next lab for correction.

Results

The motor spins forward at moderate speed for 3 seconds, stops cleanly, then spins in reverse for 3 seconds before stopping again; increasing the PWM duty cycle at a fixed direction produces a visibly faster spin, confirming correct direction control and PWM-based speed regulation through the H-bridge.

Analysis and Inferences:

Evaluator Remark (if Any):	Marks Secured: _____ out of 50
	Signature of the Evaluator with Date

Evaluator MUST ask Viva-voce prior to signing and posting marks for each experiment.

Experiment 17

Designing a UART-Based Command-Controlled Servo Motor Positioning

Aim/Objective: To combine interrupt-driven UART command reception (Experiment 11) with PWM servo control (Experiment 15) so that an operator can type a target angle on a PC terminal and have the servo move to that position.

PRE-REQUISITE:

1. Completion of Experiments 9/11 (UART reception) and Experiment 15 (servo PWM control).
2. Basic understanding of parsing numeric values from a string.

Tools & Components Required:

- STM32CubeIDE, STM32CubeMX & STM32 HAL Library
- ST Nucleo-F446RE (STM32F446RET6, Arm Cortex-M4 @ 180 MHz)
- SG90 servo motor (Experiment 15 wiring)
- PC terminal application

Description:

This experiment integrates two previously independent subsystems -- asynchronous serial command reception and hardware PWM generation -- into a single closed command-to-actuator pipeline, a pattern that recurs throughout embedded systems: parse a human- or machine-readable command, validate it, and translate it into the correct low-level peripheral register write.

Robust command parsing must validate the input range (0-180 degrees here) before applying it, since a servo driven with an out-of-range pulse width can stall against its mechanical end-stop and draw excessive current.

Circuit / Pin Connections:

Signal	Nucleo-F446RE Pin	Notes
USART2 TX/RX	PA2 / PA3	Interrupt-driven command input, 115200-8-N-1
Servo signal	PA6 (TIM3_CH1)	50 Hz PWM frame, as in Experiment 15

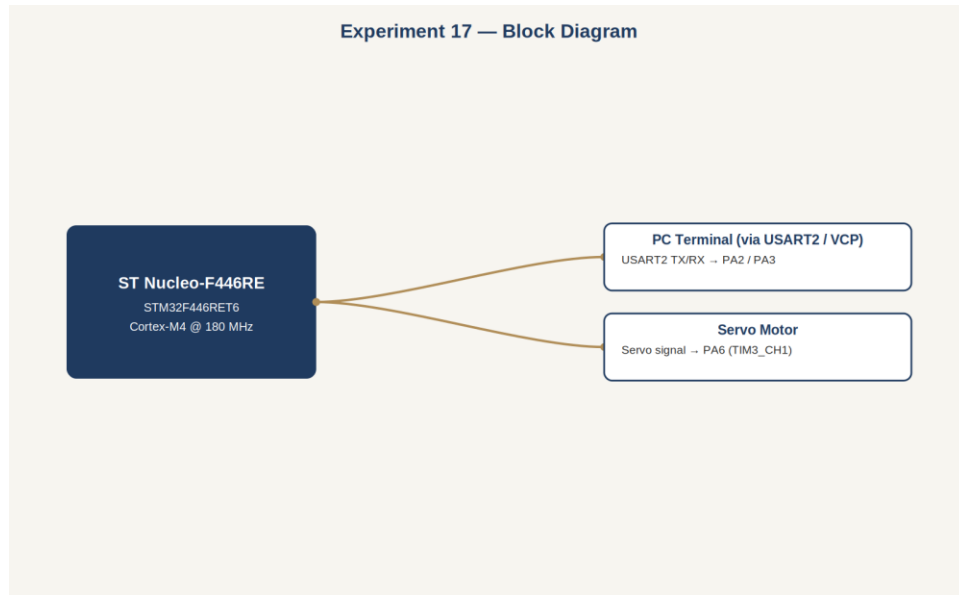


Figure: Block diagram for Experiment 17

Procedure:

Configure the required peripherals in STM32CubeMX and implement a HAL-based program to achieve the objective stated above, following the procedure and pin connections given below. Build, flash, and verify the functionality on the Nucleo-F446RE hardware.

1. Combine the interrupt-driven UART line-reception technique of Experiments 11-12 with the 50 Hz servo PWM configuration of Experiment 15.
2. Define a simple protocol: the operator types an integer angle (0-180) followed by Enter, e.g. '90\r'.
3. On receiving a complete line, convert it with `atoi()`, clamp it to `[0, 180]`, and reject (with an error message over UART) any value outside that range or any non-numeric input.
4. Map the validated angle to a pulse width in microseconds using linear interpolation: $\text{pulse_us} = 1000 + (\text{angle} * 1000) / 180$.
5. Write the computed value with `__HAL_TIM_SET_COMPARE(&tim3, TIM_CHANNEL_1, pulse_us)` and transmit a confirmation message back over UART, e.g. 'Moved to 90 deg'.
6. Flash the firmware and, from the PC terminal, test several angles (0, 45, 90, 135, 180) plus an invalid value (e.g. 200) to confirm both correct motion and correct input validation.

Program:

Viva-voce (In-Lab):

1. Why is input validation (range clamping and numeric checking) essential before writing directly to a hardware compare register?

2. Derive the linear interpolation formula used to map a 0-180 degree angle to a 1000-2000 us pulse width.

3. What would happen if this design used HAL_UART_Receive() (polling) instead of the interrupt-driven approach from Experiment 11?

4. How would you extend the protocol to support relative movement commands (e.g. '+10' to move 10 degrees from the current position)?

Post Lab Requirements:

1. Save the screenshots/pictures of the output obtained and bring the same to the next lab.
2. Note the procedures and observations as instructed by your advisor.
3. Submit the practical report in the next lab for correction.

Results

Typing a valid angle (e.g. '90') moves the servo to the corresponding position and returns a 'Moved to 90 deg' confirmation; typing an out-of-range or non-numeric value produces an 'Invalid input' response with no servo motion, confirming a correctly validated, closed command-to-actuator pipeline.

Analysis and Inferences:

Evaluator Remark (if Any):	Marks Secured: _____ out of 50
	Signature of the Evaluator with Date

Evaluator MUST ask Viva-voce prior to signing and posting marks for each experiment.

Experiment 18

Designing a Single-Channel ADC Interfacing for Potentiometer Voltage Measurement

Aim/Objective: To configure the STM32's 12-bit ADC1 on a single channel to measure the wiper voltage of a potentiometer and convert the raw digital reading into a real voltage value.

PRE-REQUISITE:

1. Completion of Experiment 1 (project setup).
2. Basic understanding of analog-to-digital conversion concepts.

Tools & Components Required:

- STM32CubeIDE, STM32CubeMX & STM32 HAL Library
- ST Nucleo-F446RE (STM32F446RET6, Arm Cortex-M4 @ 180 MHz)
- 10 kOhm linear potentiometer
- Breadboard and jumper wires

Description:

The STM32F446RE's ADC1 is a 12-bit successive-approximation converter that maps its reference voltage range (0 V to VREF+, typically 3.3 V) onto digital codes 0-4095. A potentiometer wired as a voltage divider between 3.3 V and GND presents its wiper voltage directly to an ADC input pin, providing a simple, continuously variable analog test signal.

Converting a raw ADC code to a real voltage uses: $V = (\text{raw_code} / 4095.0) \times VREF$.

HAL_ADC_GetValue() returns the raw code after a conversion completes, which

HAL_ADC_PollForConversion() waits for in the simplest (blocking/polling) usage pattern.

Circuit / Pin Connections:

Signal	Nucleo-F446RE Pin	Notes
Potentiometer wiper	PA0 (ADC1 IN0)	Analog input, 0-3.3 V
Potentiometer outer terminals	3.3V and GND	Forms the voltage-divider supply

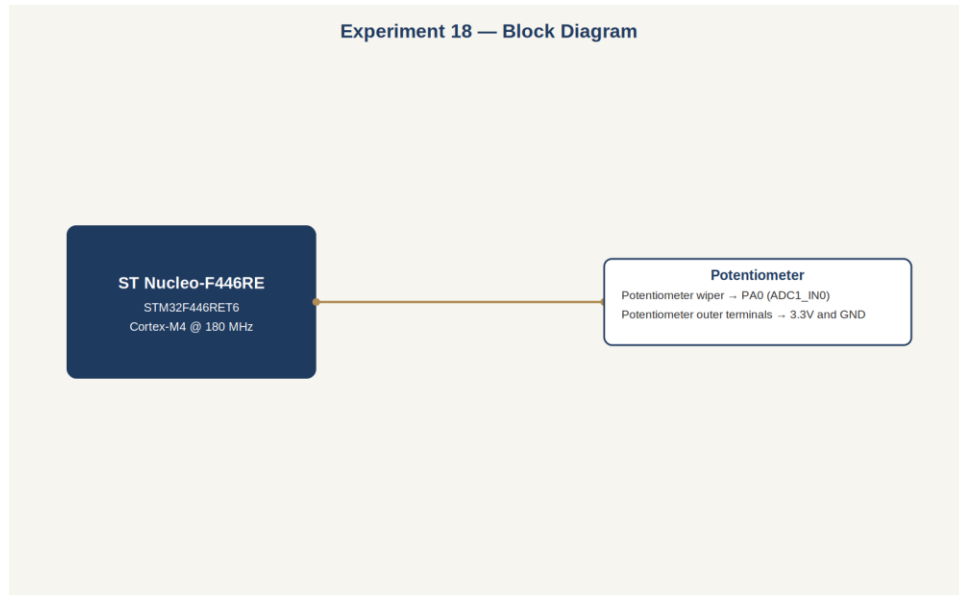


Figure: Block diagram for Experiment 18

Procedure:

Configure the required peripherals in STM32CubeMX and implement a HAL-based program to achieve the objective stated above, following the procedure and pin connections given below. Build, flash, and verify the functionality on the Nucleo-F446RE hardware.

1. Wire the potentiometer's two outer terminals to 3.3V and GND, and the wiper (center) terminal to PA0.
2. In CubeMX, enable ADC1, set IN0 as an analog channel, and configure Resolution = 12 bit, Continuous Conversion Mode as required, Sampling Time to a moderate value (e.g. 56 cycles).
3. In main.c, call `HAL_ADC_Start(&hadc1)`, then `HAL_ADC_PollForConversion(&hadc1, HAL_MAX_DELAY)`, then read `HAL_ADC_GetValue(&hadc1)`.
4. Convert the raw code to volts using $V = (\text{raw} / 4095.0f) * 3.3f$.
5. Transmit the raw code and computed voltage over UART (reusing Experiment 8) once per second for observation on the PC terminal.
6. Rotate the potentiometer through its full range and confirm the reported voltage tracks smoothly from approximately 0 V to 3.3 V.

Program:

Viva-voce (In-Lab):

1. What determines the ADC's voltage reference (VREF+) on the Nucleo-F446RE, and why does it matter for the conversion formula?

2. Why does increasing the ADC sampling time improve accuracy for high-impedance analog sources?

3. What is the theoretical voltage resolution (volts per LSB) of a 12-bit ADC referenced to 3.3 V?

4. What is the difference between single-conversion mode and continuous-conversion mode on the STM32 ADC?

Post Lab Requirements:

1. Save the screenshots/pictures of the output obtained and bring the same to the next lab.
2. Note the procedures and observations as instructed by your advisor.
3. Submit the practical report in the next lab for correction.

Results

The PC terminal shows the raw ADC code (0-4095) and computed voltage (0.00-3.30 V) updating every 500 ms, tracking the potentiometer's wiper position smoothly and monotonically across its full mechanical range, confirming correct ADC configuration and voltage-conversion math.

Analysis and Inferences:

Evaluator Remark (if Any):	Marks Secured: _____ out of 50
	Signature of the Evaluator with Date

Evaluator MUST ask Viva-voce prior to signing and posting marks for each experiment.

Experiment 19

Designing a Multi-Channel ADC Interfacing for LM35 Temperature Sensor Measurement

Aim/Objective: To interface an LM35 analog temperature sensor to a second ADC1 channel and convert the measured voltage into a temperature reading in degrees Celsius.

PRE-REQUISITE:

1. Completion of Experiment 18 (single-channel ADC interfacing).
2. Basic understanding of analog temperature sensors.

Tools & Components Required:

- STM32CubeIDE, STM32CubeMX & STM32 HAL Library
- ST Nucleo-F446RE (STM32F446RET6, Arm Cortex-M4 @ 180 MHz)
- LM35 analog temperature sensor (TO-92 package)
- Breadboard and jumper wires

Description:

The LM35 is a linear analog temperature sensor whose output voltage scales at exactly 10 mV per degree Celsius, with 0 V corresponding to 0 degC, requiring no external calibration or lookup table --
 $\text{temperature (degC)} = V_{\text{out(mV)}} / 10$.

This experiment extends Experiment 18's single-channel ADC setup to a second channel, illustrating how CubeMX's ADC channel/rank configuration scales from one analog input to several, a prerequisite for the multi-sensor experiments later in the course.

Circuit / Pin Connections:

Signal	Nucleo-F446RE Pin	Notes
LM35 VCC	3.3V (or 5V, per datasheet variant)	Sensor supply
LM35 GND	GND	Common ground
LM35 Vout	PA1 (ADC1_IN1)	Analog input, ~0-1.5 V over typical room-temperature range

Experiment 19 — Block Diagram



Figure: Block diagram for Experiment 19

Procedure:

Configure the required peripherals in STM32CubeMX and implement a HAL-based program to achieve the objective stated above, following the procedure and pin connections given below. Build, flash, and verify the functionality on the Nucleo-F446RE hardware.

1. Wire the LM35's VCC, GND, and Vout pins to 3.3V, GND, and PA1 respectively.
2. In CubeMX, add IN1 as a second ADC1 channel (Rank 1 if used alone, or Rank 2 alongside IN0 from Experiment 18 for a combined reading).
3. In main.c, perform the same Start/PollForConversion/GetValue sequence as Experiment 18 but reading channel IN1.
4. Convert the raw code to voltage ($V = \text{raw}/4095 \times 3.3$), then to millivolts, then to degrees Celsius using $\text{tempC} = V_{\text{mV}} / 10.0$.
5. Transmit the computed temperature over UART once per second.
6. Verify the reading tracks ambient temperature sensibly (approximately 20-30 degC indoors) and rises when the sensor is gently warmed (e.g. held between two fingers), then falls back as it cools.

Program:

Viva-voce (In-Lab):

1. Why does the LM35 not require any offset correction in its temperature formula, unlike some other analog sensors?

2. What error would result in the computed temperature if VREF actually measured 3.28 V rather than the assumed 3.30 V?

3. Why might averaging several consecutive ADC samples improve the LM35 reading's stability?

4. What is the LM35's typical accuracy specification, and how does that compare to the ADC's own quantization resolution at this signal range?

Post Lab Requirements:

1. Save the screenshots/pictures of the output obtained and bring the same to the next lab.
2. Note the procedures and observations as instructed by your advisor.
3. Submit the practical report in the next lab for correction.

Results

The PC terminal reports a stable temperature reading close to ambient room temperature (typically 24-28 degC), rising by a few degrees within seconds of warming the sensor and returning to baseline once released, confirming correct scaling of the LM35's 10 mV/degC output through the ADC pipeline.

Analysis and Inferences:

Evaluator Remark (if Any):	Marks Secured: _____ out of 50
	Signature of the Evaluator with Date

Evaluator MUST ask Viva-voce prior to signing and posting marks for each experiment.

Experiment 20

Multi-Channel ADC Interfacing for LDR-Based Ambient Light Measurement

Aim/Objective: To interface a Light Dependent Resistor (LDR) as a third ADC1 channel in a voltage-divider configuration and derive a relative ambient-light measurement.

PRE-REQUISITE:

1. Completion of Experiments 18-19 (ADC interfacing).
2. Basic understanding of resistive sensors and voltage dividers.

Tools & Components Required:

- STM32CubeIDE, STM32CubeMX & STM32 HAL Library
- ST Nucleo-F446RE (STM32F446RET6, Arm Cortex-M4 @ 180 MHz)
- LDR (photoresistor)
- 10 kOhm fixed resistor
- Breadboard and jumper wires

Description:

An LDR's resistance decreases as incident light increases (typically from hundreds of kilohms in darkness to a few hundred ohms in bright light). Paired with a fixed resistor in a voltage divider, the midpoint voltage therefore rises with increasing light level, which the ADC can sample just like the potentiometer in Experiment 18.

Because LDR resistance is highly nonlinear and unit-to-unit variable, this experiment reports a relative light level (raw ADC code or percentage of full scale) rather than an absolute lux value, which would require an individually characterized sensor and calibration curve.

Circuit / Pin Connections:

Signal	Nucleo-F446RE Pin	Notes
LDR + 10k divider midpoint	PA4 (ADC1_IN4)	Analog input; divider between 3.3V and GND
LDR top terminal	3.3V	Via LDR to the divider midpoint
Fixed resistor bottom terminal	GND	Via 10k resistor to the divider midpoint

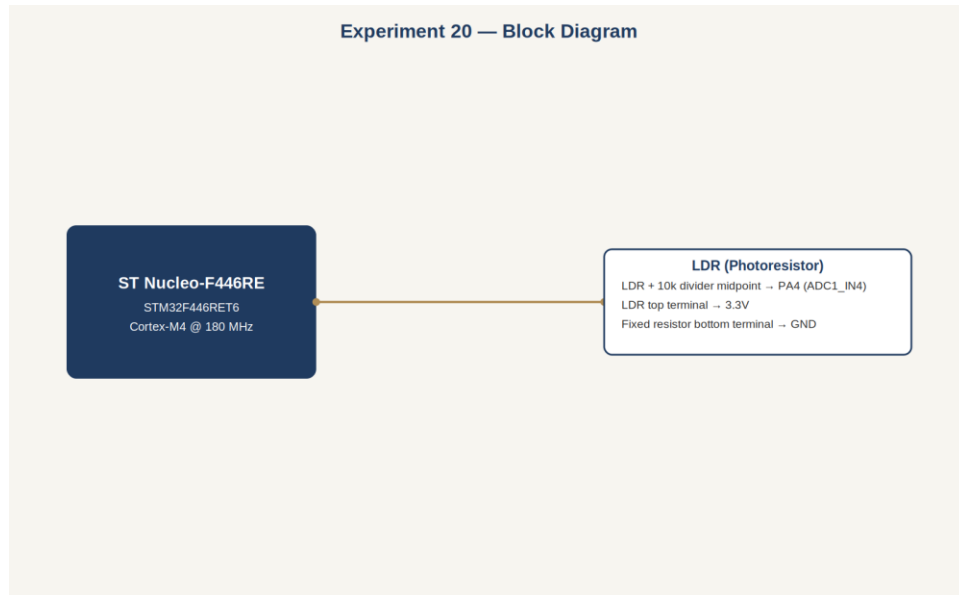


Figure: Block diagram for Experiment 20

Procedure:

Configure the required peripherals in STM32CubeMX and implement a HAL-based program to achieve the objective stated above, following the procedure and pin connections given below. Build, flash, and verify the functionality on the Nucleo-F446RE hardware.

1. Wire the LDR from 3.3V to the divider midpoint (PA4), and a 10 kOhm fixed resistor from that midpoint to GND (or the reverse arrangement, inverting the light/voltage relationship).
2. In CubeMX, add IN4 as a third ADC1 channel.
3. In main.c, sample IN4 using the same polling pattern as Experiments 18-19.
4. Convert the raw code to a percentage of full scale: $\text{lightPct} = (\text{raw} / 4095.0f) * 100.0f$.
5. Transmit the raw code and percentage over UART once per second.
6. Cover the LDR with a hand and confirm the reading changes (increases or decreases, depending on divider orientation); shine a torch/phone light on it and confirm the reading moves in the opposite direction.

Program:

Viva-voce (In-Lab):

1. Why is the reading from this circuit reported as a relative percentage rather than an absolute lux value?

2. How would swapping the LDR and fixed resistor's positions in the divider change the polarity of the light/voltage relationship?

3. What fixed-resistor value would you choose to maximize sensitivity around a specific ambient light level, and why?

4. Why is an LDR's response considered nonlinear, and what implication does that have for using it in a precise light-measurement application?

Post Lab Requirements:

1. Save the screenshots/pictures of the output obtained and bring the same to the next lab.
2. Note the procedures and observations as instructed by your advisor.
3. Submit the practical report in the next lab for correction.

Results

The reported light percentage rises when the LDR is illuminated and falls when covered (or vice versa, depending on divider orientation), tracking ambient light changes smoothly and in real time, confirming correct voltage-divider sizing and ADC sampling of the LDR.

Analysis and Inferences:

Evaluator Remark (if Any):

Marks Secured: _____ out of 50

Signature of the Evaluator with Date

Evaluator MUST ask Viva-voce prior to signing and posting marks for each experiment.

Experiment 21

Displaying LM35 Temperature and LDR Light Intensity on an I²C LCD

Aim/Objective: To simultaneously sample the LM35 (Experiment 19) and LDR (Experiment 20) ADC channels and display both readings, live, on the two rows of the I²C LCD from Experiment 7.

PRE-REQUISITE:

1. Completion of Experiments 7 (I²C LCD), 19 (LM35) and 20 (LDR).
2. Basic understanding of formatted string output (sprintf).

Tools & Components Required:

- STM32CubeIDE, STM32CubeMX & STM32 HAL Library
- ST Nucleo-F446RE (STM32F446RET6, Arm Cortex-M4 @ 180 MHz)
- LM35 sensor (Experiment 19 wiring)
- LDR voltage divider (Experiment 20 wiring)
- 16x2 I²C LCD (Experiment 7 wiring)

Description:

This experiment consolidates the two independent single-sensor experiments (19 and 20) into one multi-channel ADC scan, cycling ADC1 through channels IN1 (LM35) and IN4 (LDR) in sequence, and replaces the UART text output used previously with a persistent, human-readable I²C LCD display -- a typical structure for a simple embedded data-logging or monitoring node.

CubeMX's ADC 'Rank' mechanism allows several channels to be scanned automatically in one triggered sequence (Scan Conversion Mode = Enabled); alternatively, as used here for clarity, each channel can be selected and converted individually via HAL_ADC_ConfigChannel() immediately before each conversion.

Circuit / Pin Connections:

Signal	Nucleo-F446RE Pin	Notes
LM35 Vout	PA1 (ADC1_IN1)	As wired in Experiment 19
LDR divider midpoint	PA4 (ADC1_IN4)	As wired in Experiment 20
LCD SDA/SCL	PB9 / PB8	As wired in Experiment 7

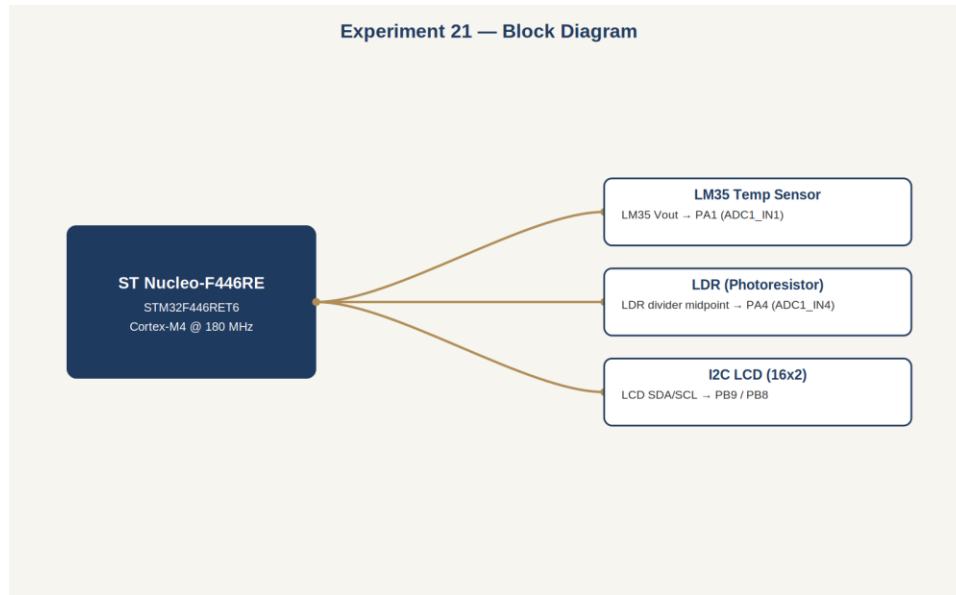


Figure: Block diagram for Experiment 21

Procedure:

Configure the required peripherals in STM32CubeMX and implement a HAL-based program to achieve the objective stated above, following the procedure and pin connections given below. Build, flash, and verify the functionality on the Nucleo-F446RE hardware.

1. Wire the LM35, LDR divider, and I2C LCD exactly as in Experiments 19, 20, and 7 respectively, all on the same project.
2. In CubeMX, enable ADC1 with both IN1 and IN4 as configured analog channels.
3. In main.c, write a helper `ReadChannel(uint32_t channel)` that calls `HAL_ADC_ConfigChannel()` to select the given channel, then performs `Start/PollForConversion/GetValue` and returns the raw code.
4. Compute `tempC` from the IN1 reading (as in Experiment 19) and `lightPct` from the IN4 reading (as in Experiment 20).
5. On every loop iteration (e.g. every 500 ms), format and print 'Temp: XX.X C' to LCD row 0 and 'Light: XX.X %' to LCD row 1.
6. Verify both readings update together and independently respond correctly to warming the LM35 and covering/illuminating the LDR.

Program:

Viva-voce (In-Lab):

1. Why is HAL_ADC_ConfigChannel() called before every conversion in this design rather than configuring both channels once at start-up?

2. What is ADC scan mode, and how could it be used here instead of manually re-selecting the channel each time?

3. What source of measurement error could arise from switching ADC channels rapidly without adequate settling/sampling time?

4. How would you extend this design to log both readings to UART simultaneously with the LCD display?

Post Lab Requirements:

1. Save the screenshots/pictures of the output obtained and bring the same to the next lab.
2. Note the procedures and observations as instructed by your advisor.
3. Submit the practical report in the next lab for correction.

Results

The LCD continuously displays both the LM35 temperature and LDR light percentage on separate rows, each updating independently and correctly in response to physically warming the sensor or changing the ambient light, confirming correct multi-channel ADC scanning and combined I2C display output.

Analysis and Inferences:

Evaluator Remark (if Any):	Marks Secured: _____ out of 50
	Signature of the Evaluator with Date

Evaluator MUST ask Viva-voce prior to signing and posting marks for each experiment.

Experiment 22

Intelligent Environmental Control: Relay Switching Based on LDR and DC Motor Speed Control Based on LM35

Aim/Objective: To build a closed-loop environmental controller that energizes a relay (e.g. simulating a lamp) when ambient light falls below a threshold, and adjusts DC motor (fan) speed proportionally to LM35 temperature.

PRE-REQUISITE:

1. Completion of Experiments 6 (relay), 16 (PWM motor) and 19-20 (LM35/LDR ADC).
2. Basic understanding of threshold-based decision making.

Tools & Components Required:

- STM32CubeIDE, STM32CubeMX & STM32 HAL Library
- ST Nucleo-F446RE (STM32F446RET6, Arm Cortex-M4 @ 180 MHz)
- LDR divider (Experiment 20)
- LM35 sensor (Experiment 19)
- 5 V relay module (Experiment 6)
- DC motor + L298N H-bridge (Experiment 16)

Description:

This experiment combines sensing (Experiments 19-20), actuation (Experiments 6 and 16), and rule evaluation (as introduced in Experiment 13) into a single autonomous environmental control system: a light-triggered relay (e.g. an automatic lamp) and a temperature-proportional fan speed controller (e.g. simple thermal management), both re-evaluated continuously against live sensor readings.

Proportional fan-speed control maps a temperature range onto a PWM duty-cycle range (e.g. 25 degC to 35 degC mapped onto 0% to 100% duty), rather than a simple on/off rule, giving smoother and more energy-efficient behaviour than the binary relay rule used for the light-triggered lamp.

Circuit / Pin Connections:

Signal	Nucleo-F446RE Pin	Notes
LDR divider midpoint	PA4 (ADC1_IN4)	As wired in Experiment 20
LM35 Vout	PA1 (ADC1_IN1)	As wired in Experiment 19
Relay IN (lamp)	PB0	As wired in Experiment 6
Motor IN1/IN2/ENA (fan)	PB1 / PB2 / PA6 (TIM3_CH1)	As wired in Experiment 16

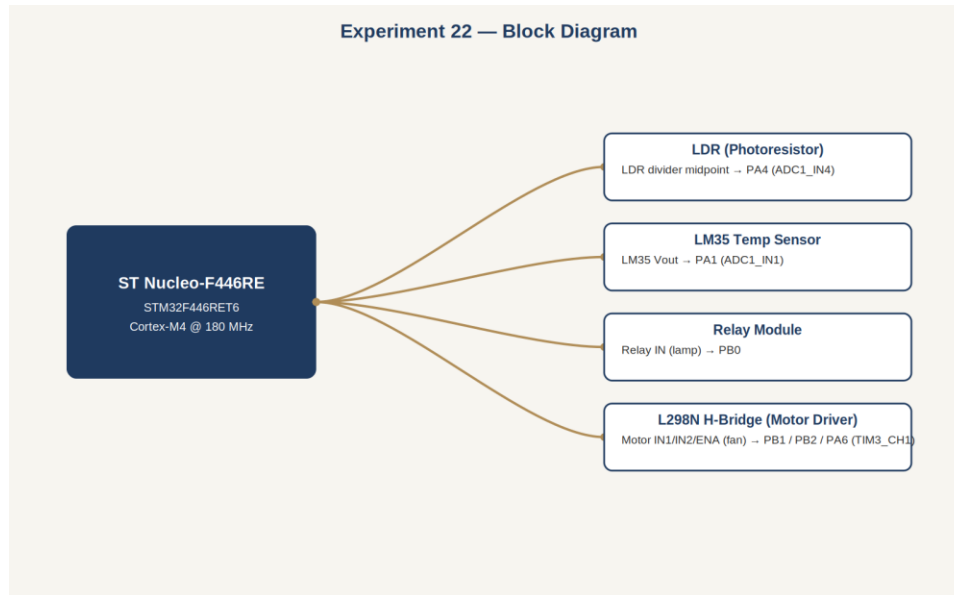


Figure: Block diagram for Experiment 22

Procedure:

Configure the required peripherals in STM32CubeMX and implement a HAL-based program to achieve the objective stated above, following the procedure and pin connections given below. Build, flash, and verify the functionality on the Nucleo-F446RE hardware.

1. Combine the LDR, LM35, relay, and DC-motor wiring from Experiments 20, 19, 6, and 16 respectively in a single project.
2. Define a light threshold (e.g. $\text{lightPct} < 30\%$ → dark) and energize the relay when dark, de-energize when bright, exactly as the AUTO rule in Experiment 13.
3. Define a temperature range (e.g. 25-35 degC) and linearly map the current LM35 reading within that range onto a PWM duty cycle 0-999 (clamped at the ends), driving `Motor_SetSpeed()` accordingly with `Motor_Forward()` held constant.
4. On every loop iteration, re-read both sensors, re-evaluate both rules, and apply both outputs -- no user command input is required once running.
5. Flash the firmware; cover the LDR and confirm the relay (lamp) switches on; warm the LM35 and confirm the fan speed increases smoothly, slowing again as it cools.
6. Log both sensor values and both actuator states over UART for observation/verification during testing.

Program:

Viva-voce (In-Lab):

1. Why is proportional control (mapping temperature to a continuous duty cycle) generally preferable to simple on/off control for fan speed?

2. What undesirable behaviour could occur at the light threshold boundary without hysteresis, and how would you add it?

3. How would you prioritize or sequence the two rules if the relay and motor shared a single, current-limited power supply?

4. What safety interlocks might a real HVAC-style controller add that this simplified experiment omits?

Post Lab Requirements:

1. Save the screenshots/pictures of the output obtained and bring the same to the next lab.
2. Note the procedures and observations as instructed by your advisor.
3. Submit the practical report in the next lab for correction.

Results

The relay (lamp) switches on automatically whenever the measured light level drops below 30%, and the fan's audible speed rises and falls smoothly as the LM35 is warmed and cooled across the 25-35 degC control band, confirming a working, continuously self-adjusting environmental controller built entirely from previously validated subsystems.

Analysis and Inferences:

Evaluator Remark (if Any):	Marks Secured: _____ out of 50
	Signature of the Evaluator with Date

Evaluator MUST ask Viva-voce prior to signing and posting marks for each experiment.

Experiment 23

Monitoring and Transmission of Sensor Data (LM35 & LDR) to PC with Actuator Status Display

Aim/Objective: To periodically transmit both LM35 temperature and LDR light readings, together with the current relay and motor actuator states from Experiment 22, to a PC terminal in a structured, loggable format.

PRE-REQUISITE:

1. Completion of Experiments 8 (UART transmit), 19-20 (ADC sensors) and 22 (actuator control).
2. Basic understanding of PC terminal applications.

Tools & Components Required:

- STM32CubeIDE, STM32CubeMX & STM32 HAL Library
- ST Nucleo-F446RE (STM32F446RET6, Arm Cortex-M4 @ 180 MHz)
- LM35, LDR, relay, and DC motor wiring (Experiment 22)
- PC terminal application or logging tool

Description:

A monitoring/telemetry layer is what turns a standalone embedded controller into an observable system: rather than relying on physical inspection, the device reports its own sensed inputs and actuated outputs over UART in a consistent, parseable format (here, comma-separated values) that a PC-side script or spreadsheet could log and graph over time.

This experiment adds no new peripheral configuration beyond Experiment 22; it demonstrates the practice of separating a system's control logic (the rules from Experiment 22) from its reporting logic (formatting and transmitting a status line), which are called on the same cycle but serve different purposes.

Circuit / Pin Connections:

Signal	Nucleo-F446RE Pin	Notes
USART2 TX	PA2	Periodic CSV status line to the PC
All Experiment 22 sensor/actuator wiring	(unchanged)	LM35: PA1, LDR: PA4, Relay: PB0, Motor: PB1/PB2/PA6

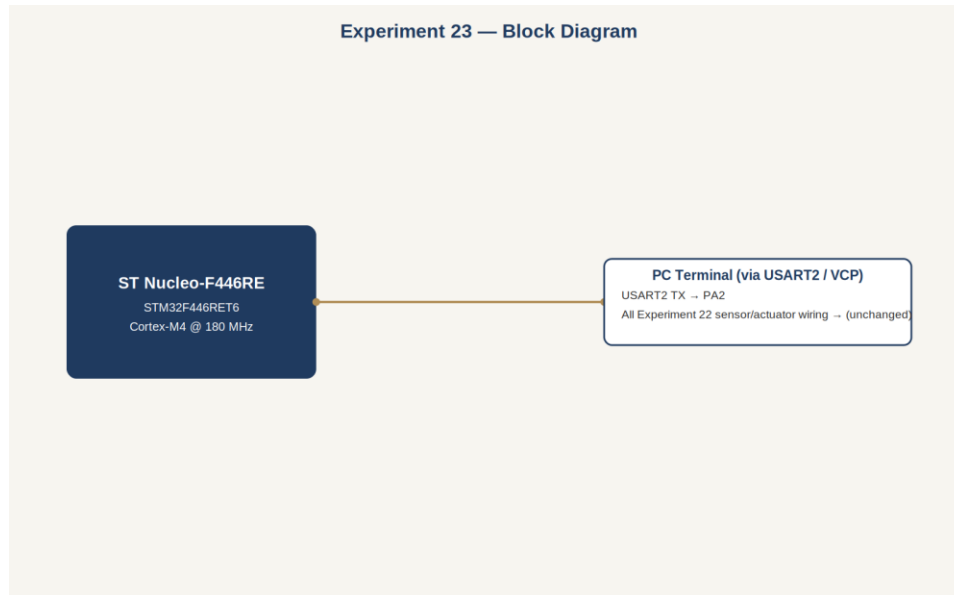


Figure: Block diagram for Experiment 23

Procedure:

Configure the required peripherals in STM32CubeMX and implement a HAL-based program to achieve the objective stated above, following the procedure and pin connections given below. Build, flash, and verify the functionality on the Nucleo-F446RE hardware.

1. Build on the completed Experiment 22 project without changing any wiring.
2. Define a CSV status format, e.g. 'TEMP,LIGHT,RELAY,FAN_DUTY\r\n' as a header printed once at start-up.
3. On every control loop iteration (after applying the Experiment 22 rules), format a data line with the current tempC, lightPct, relay state ('ON'/'OFF'), and fan duty cycle, separated by commas.
4. Transmit the line over USART2 using HAL_UART_Transmit(), once per control cycle (e.g. every 500 ms - 1 s).
5. On the PC, capture the terminal output to a text/log file (most terminal apps support logging) for later import into a spreadsheet.
6. Verify that manually changing the ambient light and temperature produces corresponding, correctly time-aligned changes in all four logged fields.

Program:

Viva-voce (In-Lab):

1. Why is a fixed, machine-parseable format (such as CSV) preferable to free-form human-readable text for a telemetry stream intended for logging?

2. How would you extend this design to also accept configuration commands from the PC (bidirectional telemetry, as previewed in Experiment 13) without interfering with the periodic status output?

3. What are the practical limits on how frequently status lines can be transmitted at 115200 baud before the link becomes saturated?

4. How would you add a timestamp field to each logged line, and what time source would you use?

Post Lab Requirements:

1. Save the screenshots/pictures of the output obtained and bring the same to the next lab.
2. Note the procedures and observations as instructed by your advisor.
3. Submit the practical report in the next lab for correction.

Results

The PC terminal receives a continuous stream of comma-separated status lines (temperature, light, relay state, fan duty) every 500 ms, correctly reflecting live sensor changes and matching the actuator behaviour observed directly on the hardware, confirming a complete and correctly synchronized monitoring pipeline.

Analysis and Inferences:

Evaluator Remark (if Any):	Marks Secured: _____ out of 50
	Signature of the Evaluator with Date

Evaluator MUST ask Viva-voce prior to signing and posting marks for each experiment.

Experiment 24

Developing a CAN Peripheral Initialization and Loopback Communication Test

Aim/Objective: To configure the STM32F446RE's bxCAN peripheral in internal Loopback mode and verify correct transmit/receive operation entirely in software, without any external transceiver or second node.

PRE-REQUISITE:

1. Completion of Experiment 1 (project setup).
2. Basic understanding of serial bus communication (as covered in the UART experiments).

Tools & Components Required:

- STM32CubeIDE, STM32CubeMX & STM32 HAL Library
- ST Nucleo-F446RE (STM32F446RET6, Arm Cortex-M4 @ 180 MHz)
- USB cable

Description:

CAN (Controller Area Network) is a robust, multi-master, message-based serial bus widely used in automotive and industrial systems, using differential signalling (CAN_H, CAN_L) for high noise immunity and built-in arbitration so that multiple nodes can share a single bus without a central controller.

The STM32F446RE's bxCAN peripheral supports a Loopback mode in which transmitted messages are internally routed back to the receive side without needing a physical bus, transceiver, or second node -- an efficient way to verify filter configuration, message framing, and the TX/RX software path before committing to real multi-node hardware in Experiments 25-26.

Circuit / Pin Connections:

Signal	Nucleo-F446RE Pin	Notes
CAN1_TX	PA12	Not required for pure Loopback mode, but configured for later experiments
CAN1_RX	PA11	Not required for pure Loopback mode, but configured for later experiments

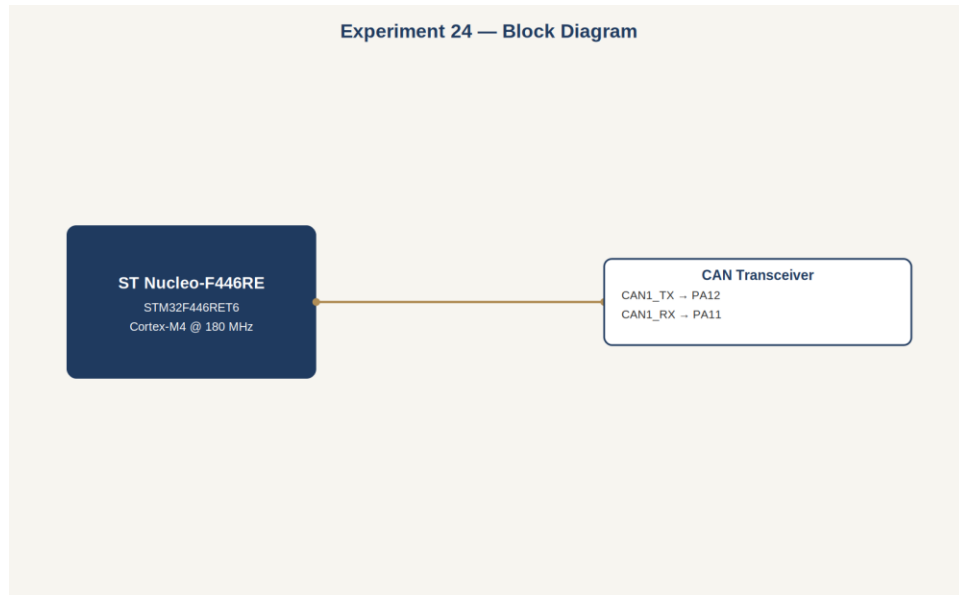


Figure: Block diagram for Experiment 24

Procedure:

Configure the required peripherals in STM32CubeMX and implement a HAL-based program to achieve the objective stated above, following the procedure and pin connections given below. Build, flash, and verify the functionality on the Nucleo-F446RE hardware.

1. In CubeMX, enable CAN1 and set Mode = 'Loopback' (Internal Loopback, no external transceiver needed).
2. Configure the bit timing (Prescaler, BS1, BS2) for a standard bus speed, e.g. 500 kbit/s at the given APB1 clock.
3. Configure at least one CAN filter (Filter Bank 0) in CubeMX to accept all standard IDs, and assign it to FIFO0.
4. Generate code; in main.c, call `HAL_CAN_Start(&hcan1)` and `HAL_CAN_ActivateNotification()` for the FIFO0 message-pending interrupt (or use polling with `HAL_CAN_GetRxMessage()`).
5. Build a `CAN_TxHeaderTypeDef` with a test Standard ID (e.g. 0x100), DLC = 2, and a 2-byte payload; transmit it with `HAL_CAN_AddTxMessage()`.
6. Poll `HAL_CAN_GetRxFifoFillLevel()` and then `HAL_CAN_GetRxMessage()` to retrieve the looped-back frame; compare the received ID and payload against what was sent, and toggle LD2 on a successful match.

Program:

Viva-voce (In-Lab):

1. What is the purpose of testing a CAN peripheral in Loopback mode before connecting real bus hardware?

2. How is the CAN bit rate determined from the Prescaler, BS1, and BS2 timing parameters?

3. What role does a CAN acceptance filter play, and what would happen if the configured filter did not match the test message's ID?

4. What is the difference between CAN Standard ID (11-bit) and Extended ID (29-bit) addressing?

Post Lab Requirements:

1. Save the screenshots/pictures of the output obtained and bring the same to the next lab.
2. Note the procedures and observations as instructed by your advisor.
3. Submit the practical report in the next lab for correction.

Results

LD2 turns on immediately after the loopback transmission, confirming the received Standard ID and payload bytes exactly match what was transmitted -- validating the CAN peripheral's bit-timing configuration, filter setup, and TX/RX software path before any external hardware is introduced.

Analysis and Inferences:

Evaluator Remark (if Any):	Marks Secured: _____ out of 50
	Signature of the Evaluator with Date

Evaluator MUST ask Viva-voce prior to signing and posting marks for each experiment.

Experiment 25

Developing a CAN Communication Between Two STM32 Nucleo Boards

Aim/Objective: To establish real two-node CAN bus communication between two Nucleo-F446RE boards, each fitted with an external CAN transceiver, exchanging messages over a physical differential bus.

PRE-REQUISITE:

1. Completion of Experiment 24 (CAN peripheral loopback test).
2. Basic understanding of multi-node bus arbitration.

Tools & Components Required:

- STM32CubeIDE, STM32CubeMX & STM32 HAL Library
- ST Nucleo-F446RE (STM32F446RET6, Arm Cortex-M4 @ 180 MHz) (x2, one per node)
- CAN transceiver module (e.g. SN65HVD230 or TJA1050), one per board
- 120 ohm termination resistors (x2, one at each end of the bus)
- Twisted-pair wiring for CAN_H/CAN_L
- Jumper wires

Description:

Unlike Experiment 24's internal loopback, the STM32's bxCAN peripheral produces logic-level TX/RX signals that must be converted to the CAN bus's differential CAN_H/CAN_L signalling by an external transceiver IC; the transceiver also provides the necessary bus fault protection.

A correctly terminated CAN bus requires a 120 ohm resistor at each physical end of the bus (not at every node) to prevent signal reflections; omitting termination is one of the most common causes of unreliable or completely non-functional CAN communication in a lab setup.

Circuit / Pin Connections:

Signal	Nucleo-F446RE Pin	Notes
CAN1_TX (each board)	PA12 -> transceiver TXD	Logic-level TX into the transceiver
CAN1_RX (each board)	PA11 <- transceiver RXD	Logic-level RX from the transceiver
Transceiver CAN_H / CAN_L	Bus wiring between both nodes	Differential pair, 120 ohm termination at each physical end
Transceiver VCC/GND	3.3V or 5V (per transceiver) / GND	Common ground required between both nodes

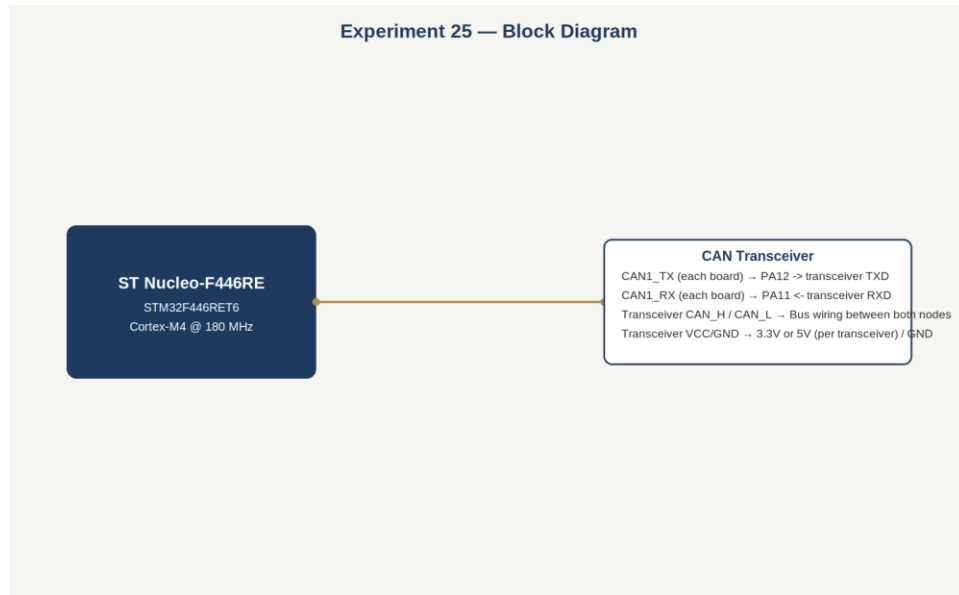


Figure: Block diagram for Experiment 25

Procedure:

Configure the required peripherals in STM32CubeMX and implement a HAL-based program to achieve the objective stated above, following the procedure and pin connections given below. Build, flash, and verify the functionality on the Nucleo-F446RE hardware.

1. On each Nucleo board, wire PA12/PA11 to the transceiver's TXD/RXD, power the transceiver, and connect its CAN_H/CAN_L pins to the shared bus wiring; fit a 120 ohm resistor at each physical end of the CAN_H/CAN_L pair.
2. In CubeMX on both projects, configure CAN1 in Normal mode (not Loopback) at a matching bit rate (e.g. 500 kbit/s) on both boards, with a filter accepting all Standard IDs.
3. On Board A, periodically transmit a message with ID 0x200 containing an incrementing counter byte, using HAL_CAN_AddTxMessage() as in Experiment 24.
4. On Board B, poll (or use interrupt notification for) HAL_CAN_GetRxMessage() and print each received ID and payload over its own USART2 to a PC terminal.
5. Flash Board A's transmitter firmware and Board B's receiver firmware, power both boards, and confirm Board B's terminal shows the incrementing counter arriving reliably.
6. Swap roles (or make both boards transmit and receive) to confirm bidirectional communication and correct arbitration when both transmit near-simultaneously.

Program:

Viva-voce (In-Lab):

1. Why is external termination (120 ohm at each physical bus end) required for real CAN communication but not for the internal Loopback mode of Experiment 24?

2. What happens during CAN bus arbitration if two nodes begin transmitting different message IDs at exactly the same time?

3. Why must both nodes' bxCAN bit-timing configuration produce the same effective bit rate, even if their Prescaler/BS1/BS2 values differ?

4. What is the function of the external CAN transceiver IC that the STM32's own bxCAN peripheral cannot perform on its own?

Post Lab Requirements:

1. Save the screenshots/pictures of the output obtained and bring the same to the next lab.
2. Note the procedures and observations as instructed by your advisor.
3. Submit the practical report in the next lab for correction.

Results

Board B's PC terminal displays the incrementing counter value transmitted by Board A, arriving reliably at the expected ~200 ms interval with the correct Standard ID (0x200), confirming successful physical-layer CAN communication between two independent nodes including transceiver wiring and bus termination.

Analysis and Inferences:

Evaluator Remark (if Any):	Marks Secured: _____ out of 50
	Signature of the Evaluator with Date

Evaluator MUST ask Viva-voce prior to signing and posting marks for each experiment.

Experiment 26

Designing a Sensor Data Acquisition and Transmission Over CAN Bus

Aim/Objective: To acquire live LM35 temperature and LDR light sensor data on one Nucleo board and transmit it over the two-node CAN bus from Experiment 25 to a second board, which displays the received readings on its I2C LCD.

PRE-REQUISITE:

1. Completion of Experiments 19-20 (ADC sensor reading) and 25 (two-node CAN communication).
2. Basic understanding of packing sensor data into fixed-size message fields.

Tools & Components Required:

- STM32CubeIDE, STM32CubeMX & STM32 HAL Library
- ST Nucleo-F446RE (STM32F446RET6, Arm Cortex-M4 @ 180 MHz) (x2)
- CAN transceivers and bus wiring (Experiment 25)
- LM35 and LDR sensors (Experiments 19-20), wired to the transmitting board
- 16x2 I2C LCD (Experiment 7), wired to the receiving board

Description:

This capstone experiment integrates ADC sensor acquisition (Experiments 19-20), CAN messaging (Experiment 25), and I2C display output (Experiment 7) into a distributed two-node system: one node acts purely as a sensor/transmitter, the other purely as a display/receiver, communicating only over the shared CAN bus -- a structure directly reflective of real distributed automotive/industrial sensor networks.

Because a single CAN data frame carries at most 8 payload bytes, the two sensor readings are packed into a compact fixed-point representation (e.g. temperature x10 and light-percentage x10 as two 16-bit integers) rather than transmitted as raw floating-point text, illustrating a realistic CAN payload-encoding technique.

Circuit / Pin Connections:

Signal	Nucleo-F446RE Pin	Notes
LM35 Vout (Node A)	PA1 (ADC1_IN1)	As wired in Experiment 19, on the transmitting board
LDR divider (Node A)	PA4 (ADC1_IN4)	As wired in Experiment 20, on the transmitting board
CAN1 TX/RX (both nodes)	PA12 / PA11	As wired in Experiment 25
LCD SDA/SCL (Node B)	PB9 / PB8	As wired in Experiment 7, on the receiving board

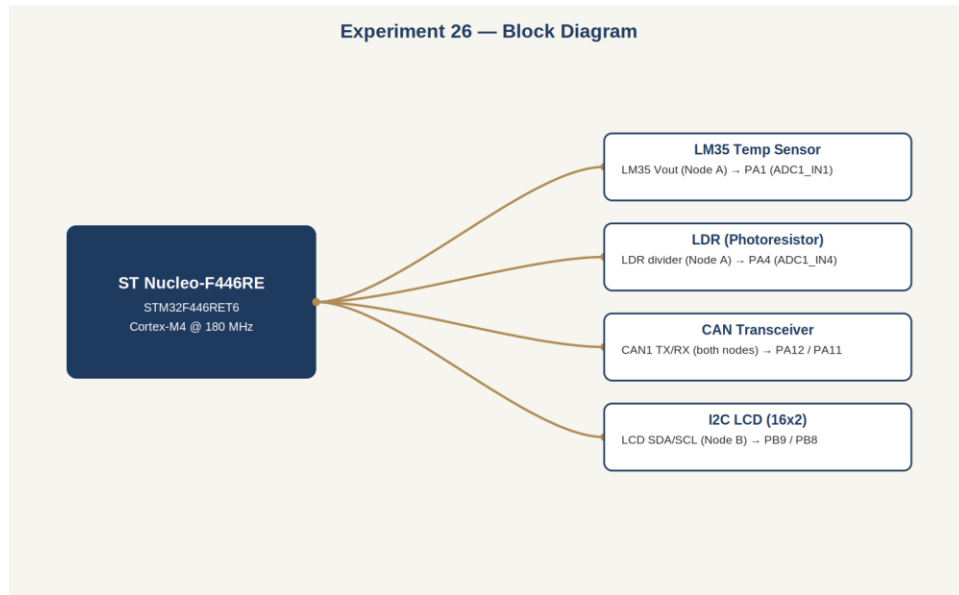


Figure: Block diagram for Experiment 26

Procedure:

Configure the required peripherals in STM32CubeMX and implement a HAL-based program to achieve the objective stated above, following the procedure and pin connections given below. Build, flash, and verify the functionality on the Nucleo-F446RE hardware.

1. On Node A (transmitter), combine the LM35/LDR ADC reading code from Experiments 19-20 with the CAN transmitter code from Experiment 25.
2. Pack tempC and lightPct into two int16_t fields as fixed-point values scaled by 10 (e.g. 27.4 degC -> 274), and copy both into an 8-byte CAN payload (4 bytes used, DLC = 4) with StdId = 0x300.
3. Transmit this frame once per second using HAL_CAN_AddTxMessage(), as in Experiment 25.
4. On Node B (receiver), combine the CAN receiver code from Experiment 25 with the I2C LCD driver from Experiment 7.
5. On receiving a frame with StdId == 0x300, unpack the two int16_t fields, divide by 10.0f to recover tempC and lightPct, and print them to LCD rows 0 and 1 respectively.
6. Power both boards simultaneously and confirm Node B's LCD updates once per second with temperature and light values that correctly track physical changes applied at Node A's sensors.

Program:

Viva-voce (In-Lab):

1. Why are the sensor readings encoded as scaled 16-bit integers rather than transmitted as raw floating-point values in the CAN payload?

2. What is the maximum data payload size of a classic CAN 2.0 data frame, and how does that constrain the design of a sensor-telemetry protocol?

3. How would you extend this system to support additional sensor nodes broadcasting on the same bus without ID collisions?

4. What would happen to Node B's displayed values if Node A's CAN frame were lost or corrupted on the bus, and how could the protocol be made more robust against that?

Post Lab Requirements:

1. Save the screenshots/pictures of the output obtained and bring the same to the next lab.
2. Note the procedures and observations as instructed by your advisor.
3. Submit the practical report in the next lab for correction.

Results

Node B's LCD updates once per second with temperature and light readings that correctly and promptly track physical changes applied to Node A's sensors, confirming a fully working distributed sensor-acquisition-and-display system across the CAN bus, with correct fixed-point payload packing/unpacking on both ends.

Analysis and Inferences:

Evaluator Remark (if Any):	Marks Secured: _____ out of 50
	Signature of the Evaluator with Date

Evaluator MUST ask Viva-voce prior to signing and posting marks for each experiment.